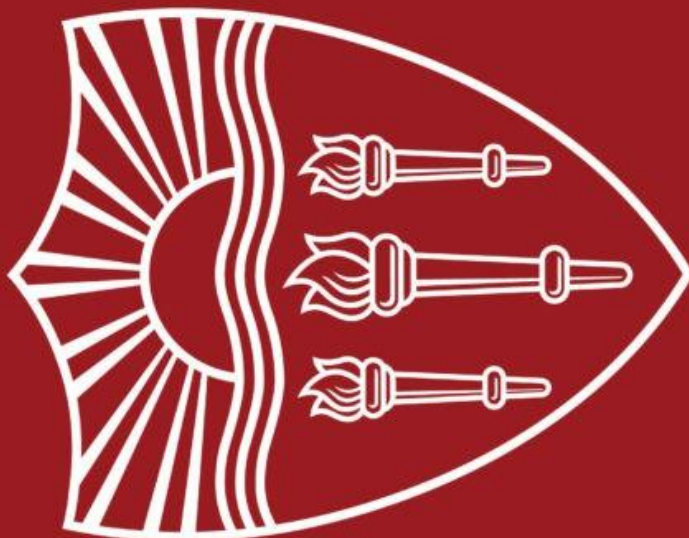




Lecture 10: Sequence-to-Sequence Modeling with Encoder-Decoder Architectures

Instructor: Xiang Ren
USC CSCI 544 Applied NLP
Fall 2026

Paper Review Report

- **Individual report; two-page summary of one/multiple papers on a topic**
- **Options:**
 1. Literature review of the course project – a few most relevant papers
 2. The course explores topics through a series of assigned readings in the form of book chapters.
- **Deadline: Oct 6**

Announcements

- **One recently enrolled student is looking to join a project team (to make a 7-people team)!**
- **Check Piazza if your team is interested in taking one more member**

Recap: Recurrent Neural Nets (RNNs)

Recurrent Neural Net Language Models

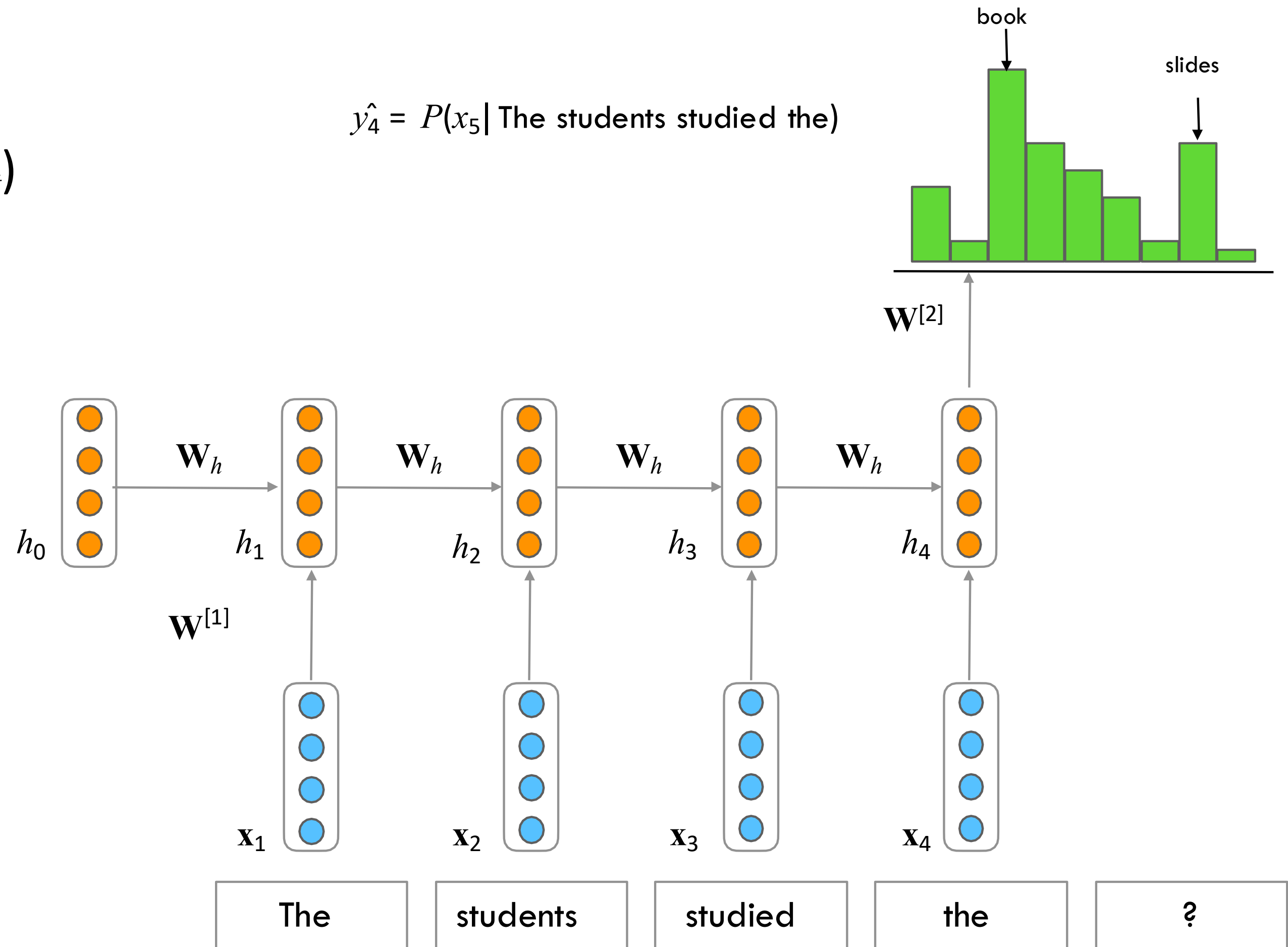
Output layer: $\hat{y}_t = \text{softmax}(\mathbf{W}^{[2]}\mathbf{h}_t)$

Hidden layer:

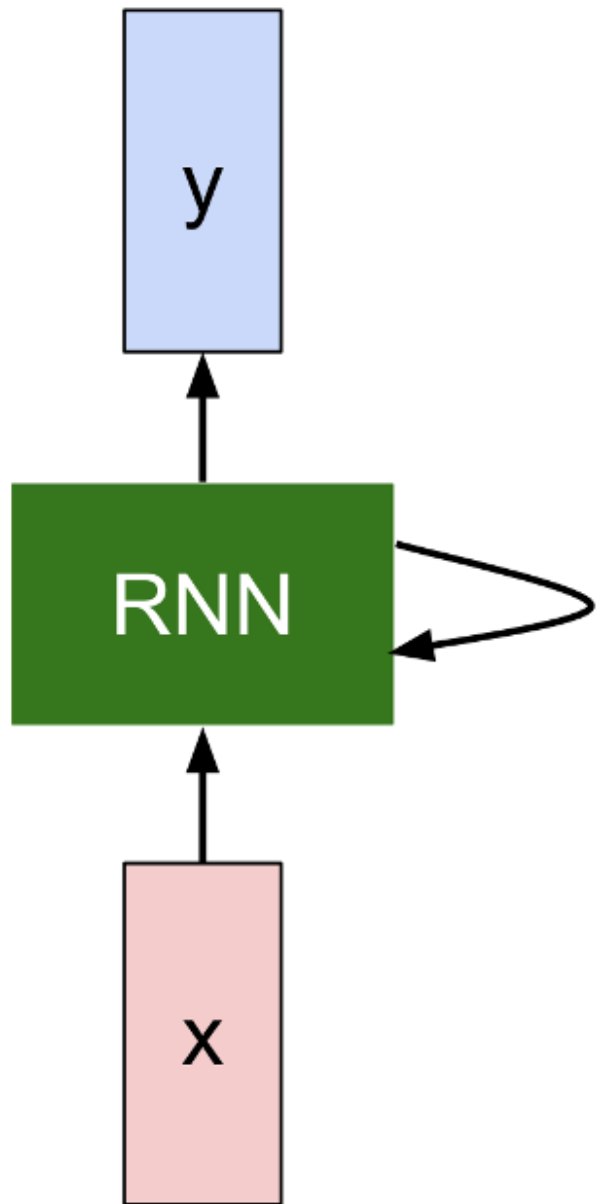
$$\mathbf{h}_t = g(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}^{[1]} \mathbf{x}_t)$$

Initial hidden state: $\mathbf{h}_0$

Word Embeddings, $\mathbf{x}_i$



RNN

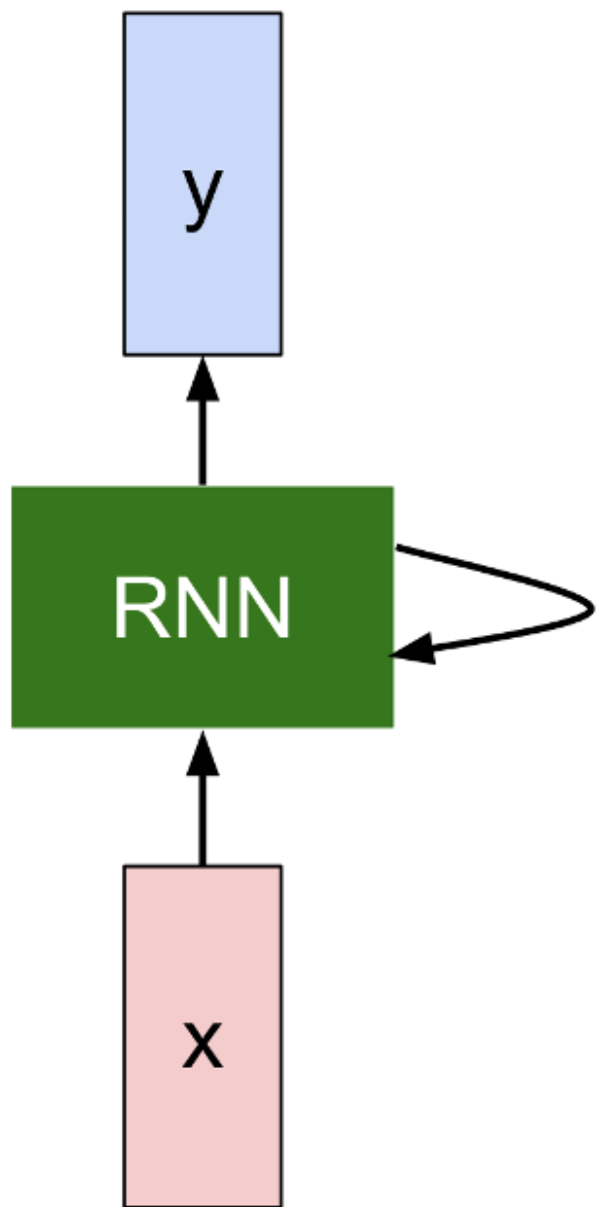


We can process a sequence of vectors $\mathbf{x}$ by applying a **recurrence formula** at every time step:

$$h_t = f_W(h_{t-1}, x_t)$$

Note: the same function with the same parameters

(Vanilla) RNN



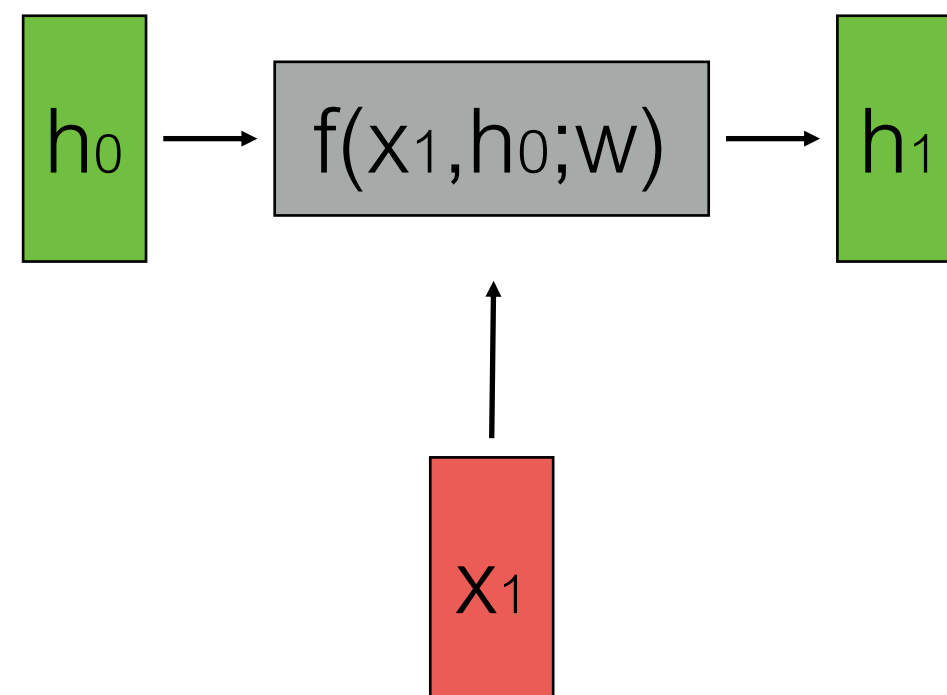
$$h_t = f_W(h_{t-1}, x_t)$$



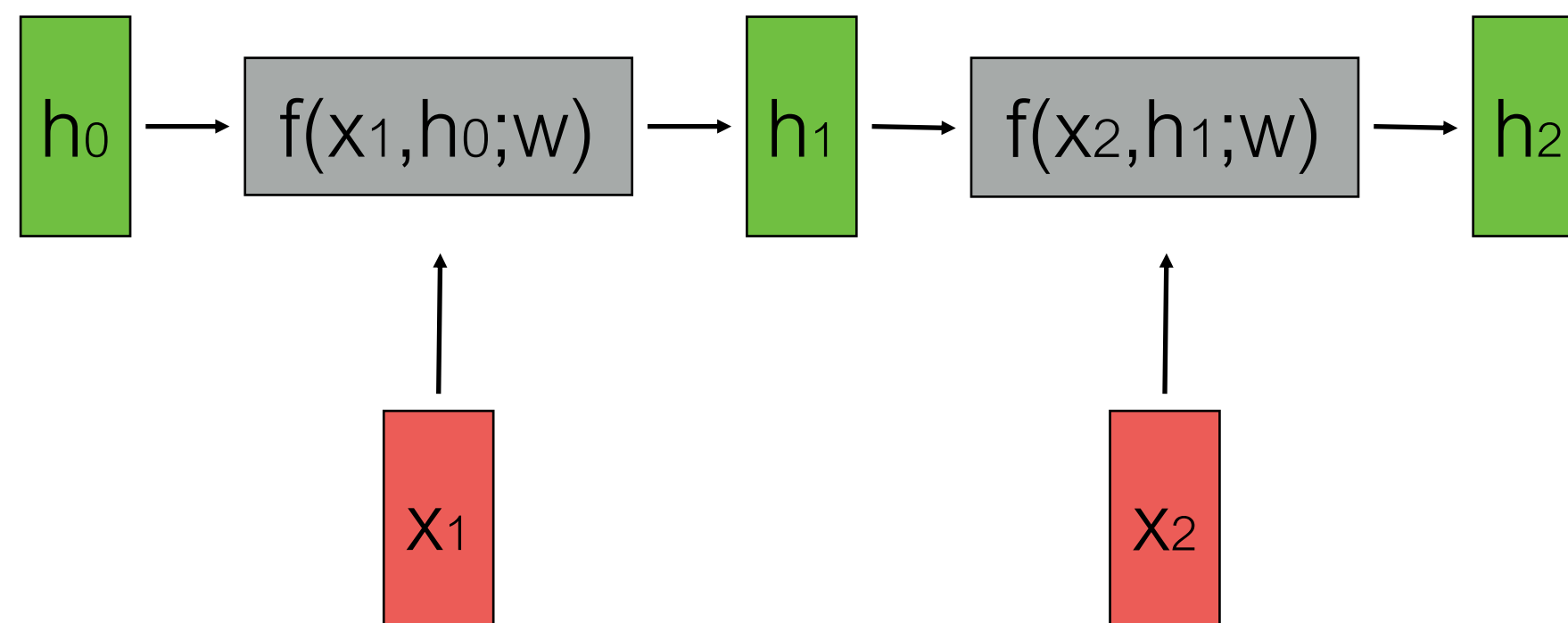
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

$$y_t = W_{hy}h_t$$

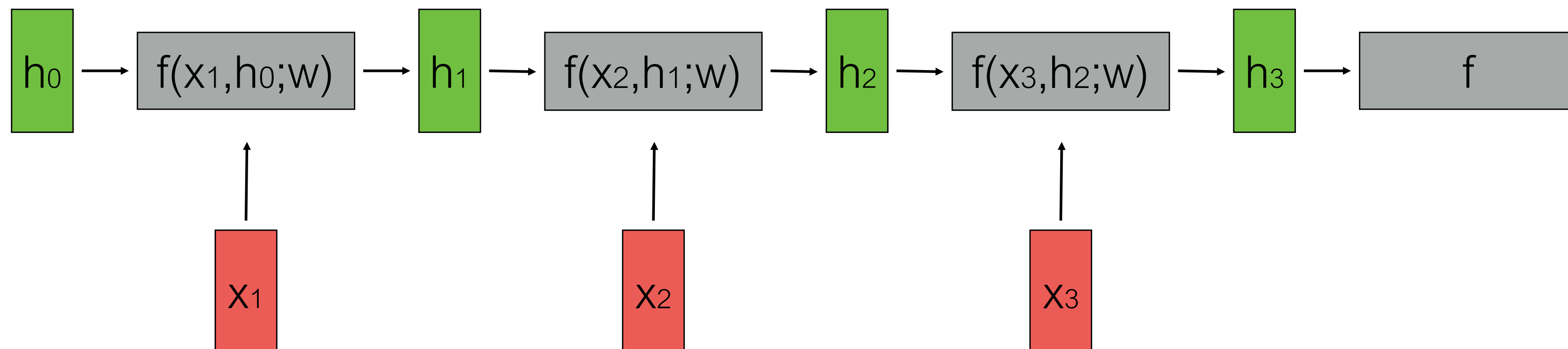
Computational Graph for RNN



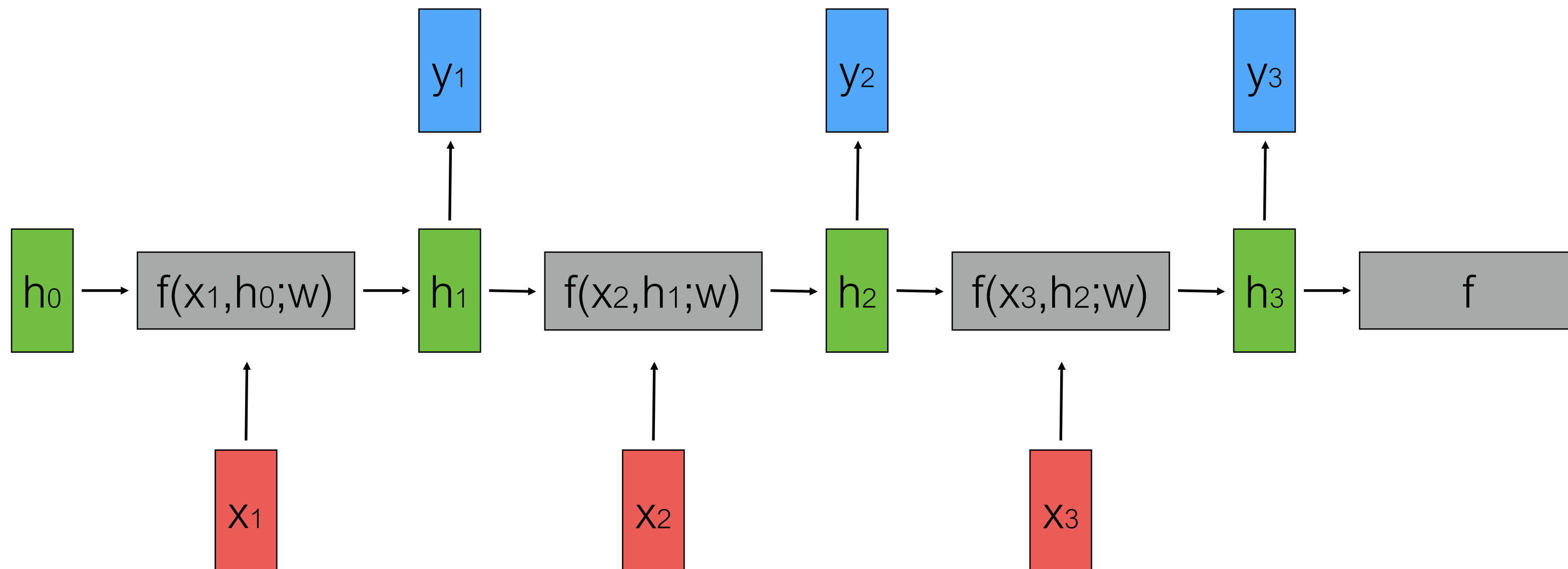
Computational Graph for RNN



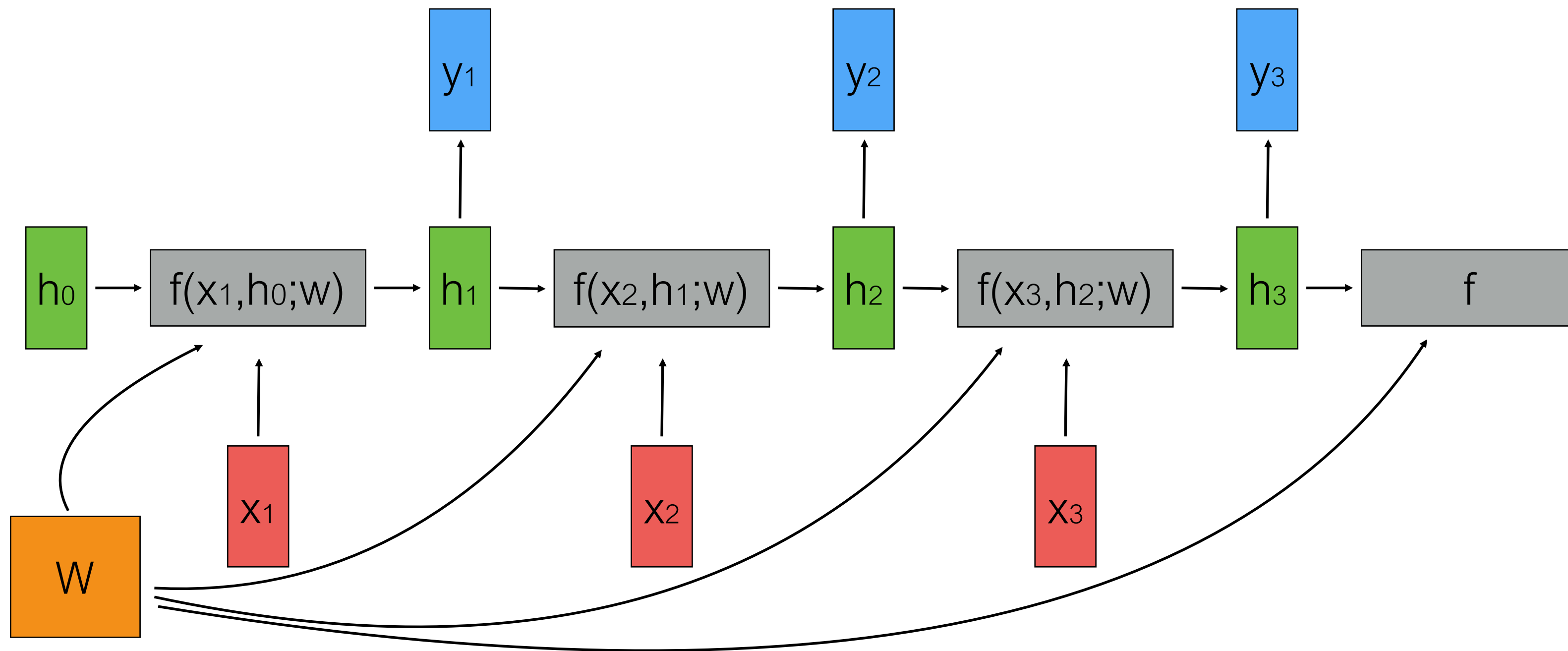
Computational Graph for RNN



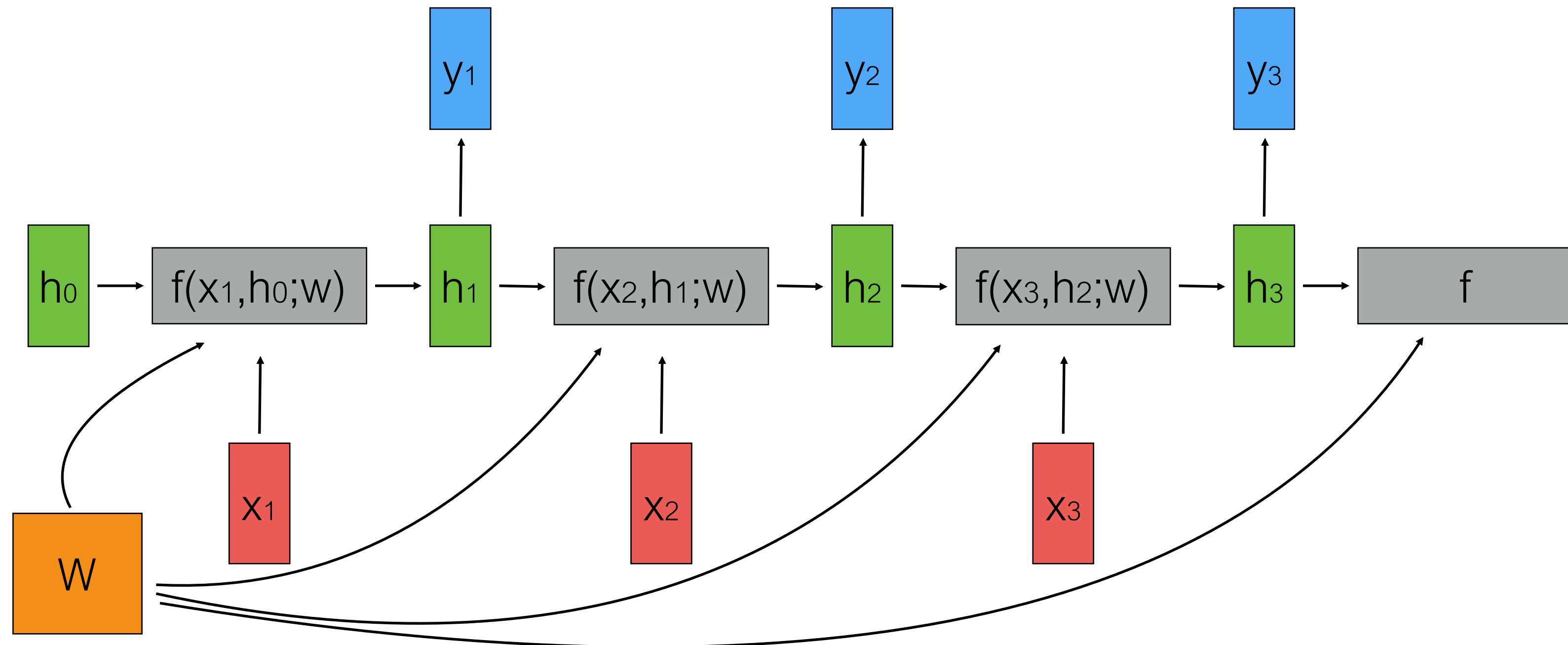
Computational Graph for RNN



Computational Graph for RNN

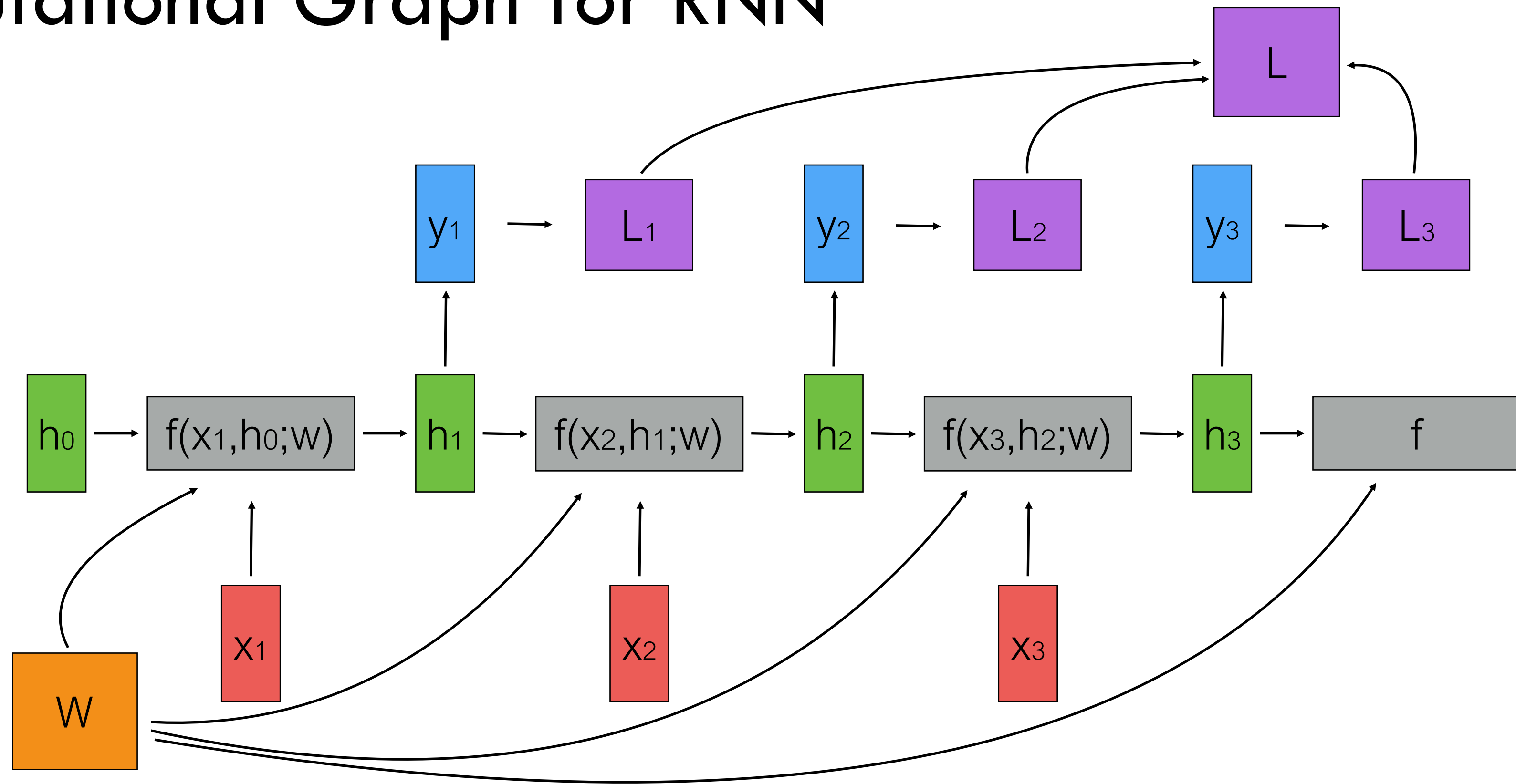


Computational Graph for RNN



The same weight (w) is shared across all f

Computational Graph for RNN

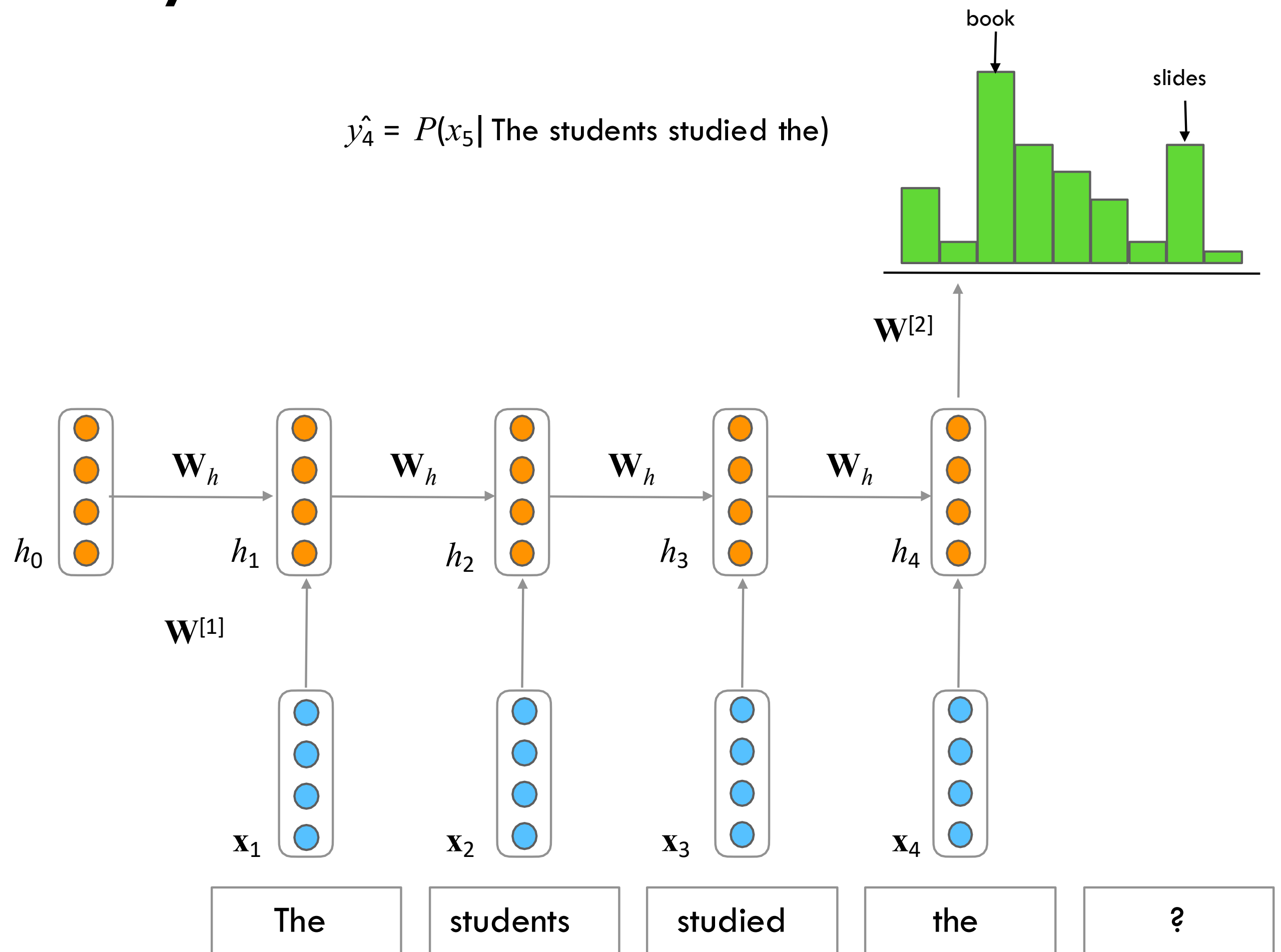


The same weight (w) is shared across all f

Why RNNs?

RNN Advantages:

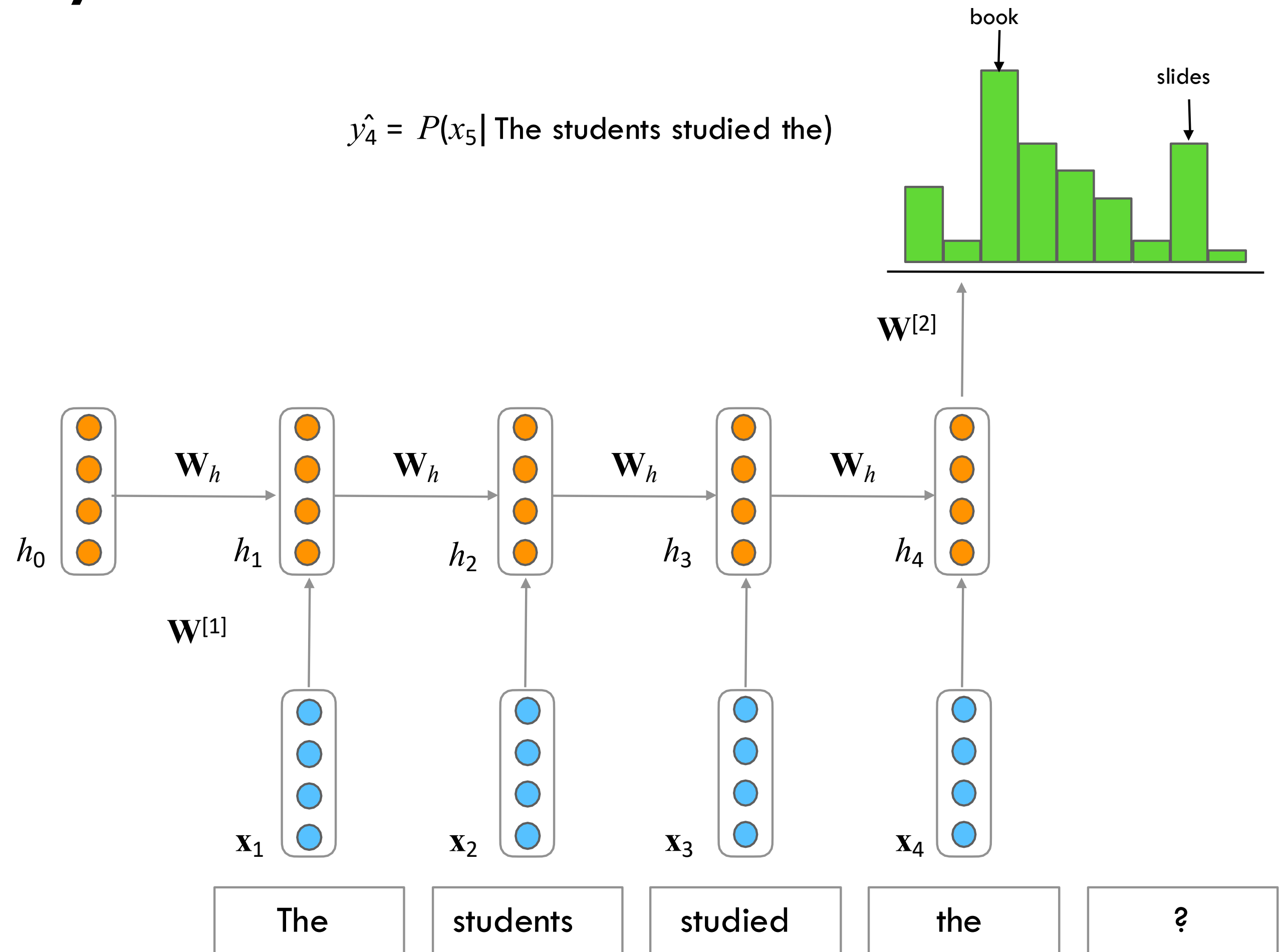
- Can process any length input
- Model size doesn't increase for longer input
- Computation for step t can (in theory) use information from many steps back
- Weights $\mathbf{W}^{[1]}$ are shared (tied) across timesteps $\rightarrow$ Condition the neural network on all previous words



Why not RNNs?

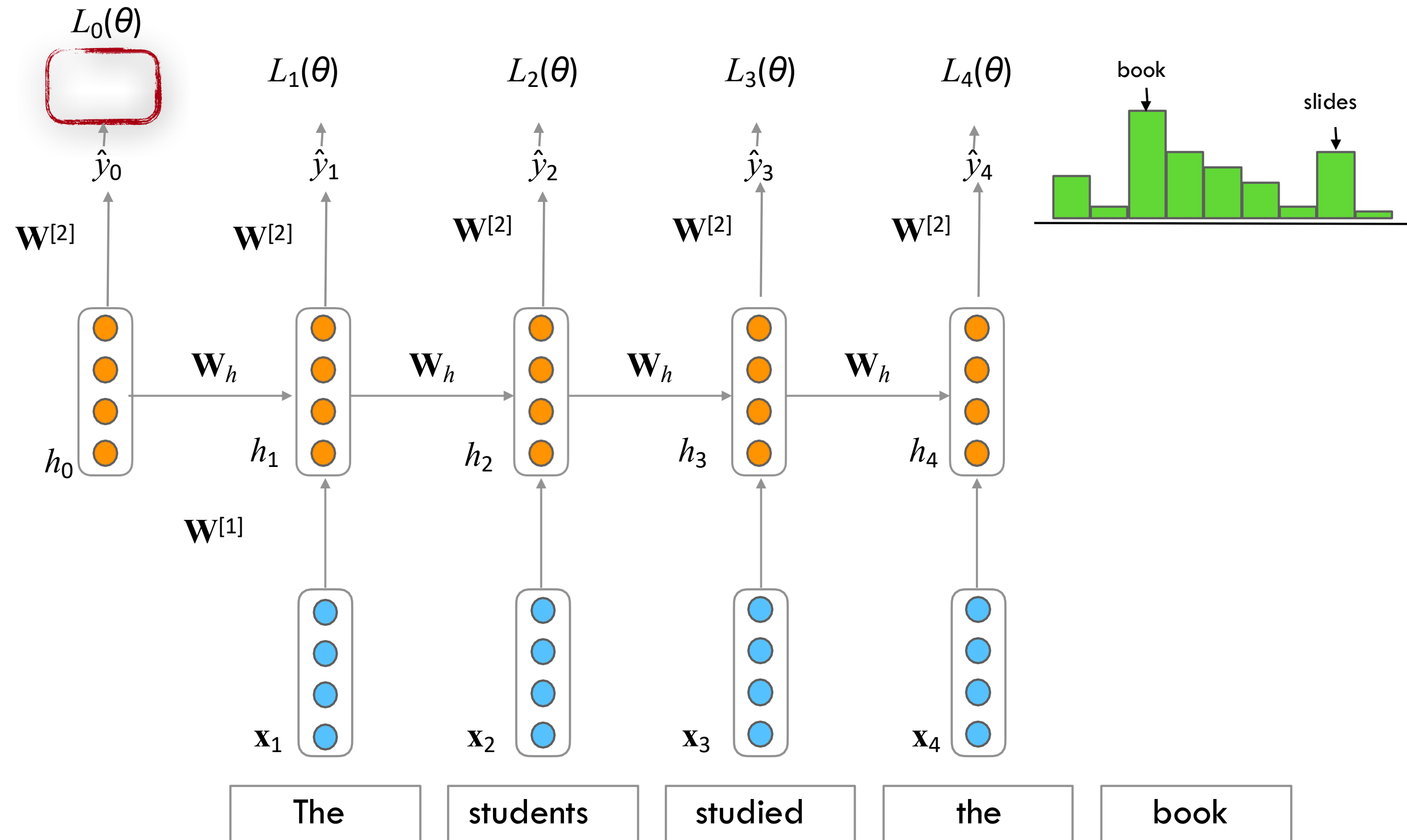
RNN Disadvantages:

- Recurrent computation is slow
- In practice, difficult to access information from many steps back

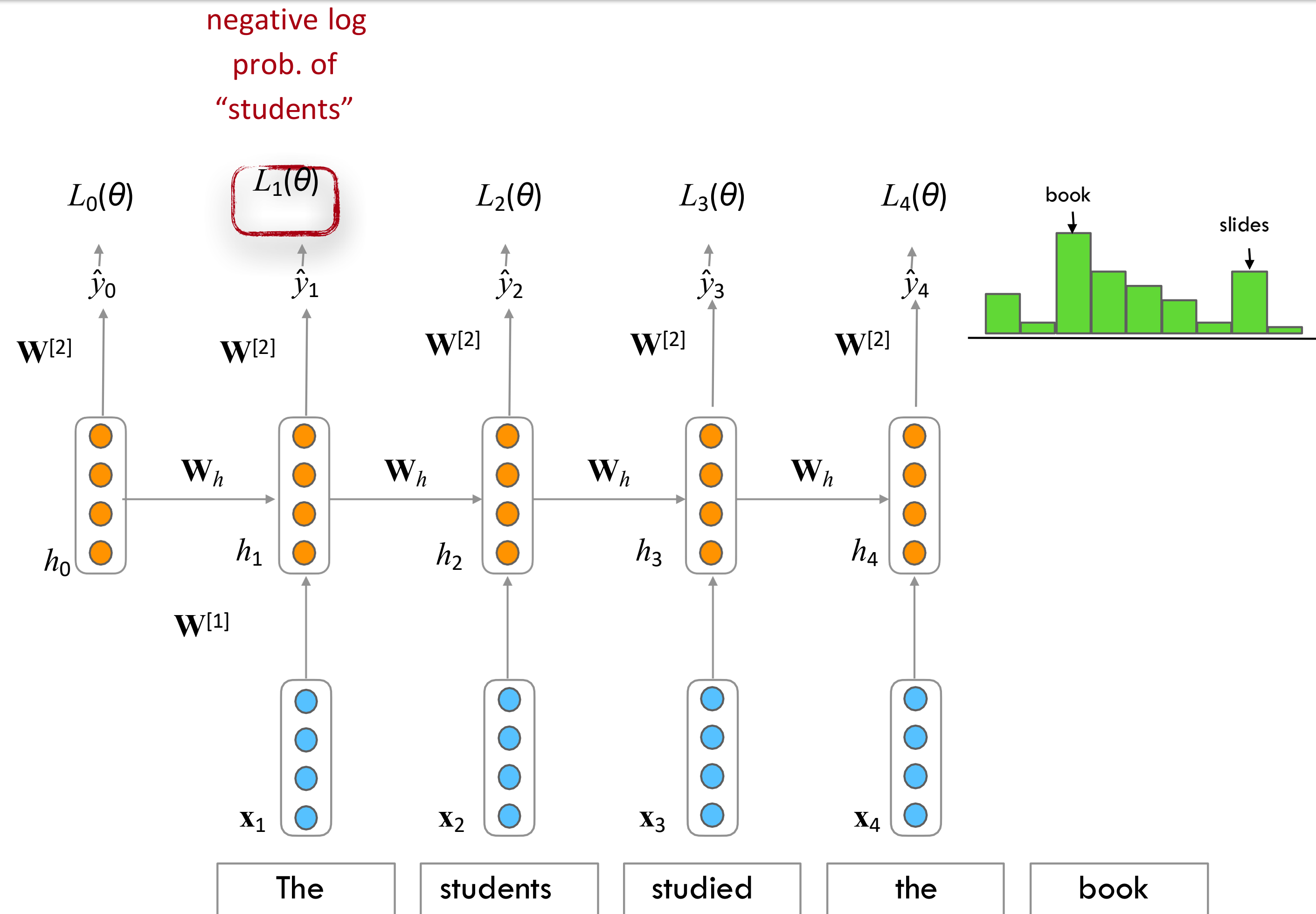


negative log
prob. of "The"

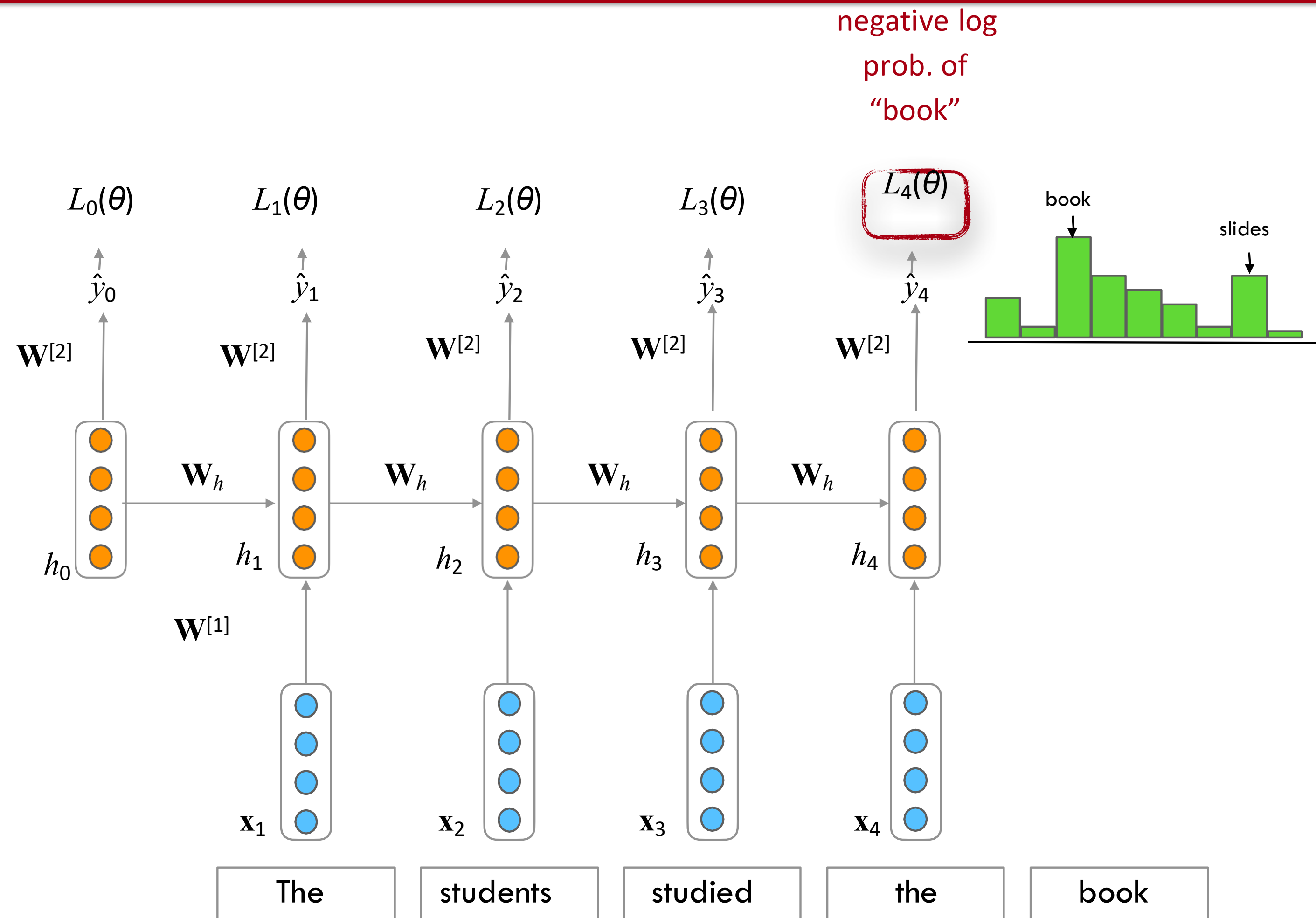
Loss



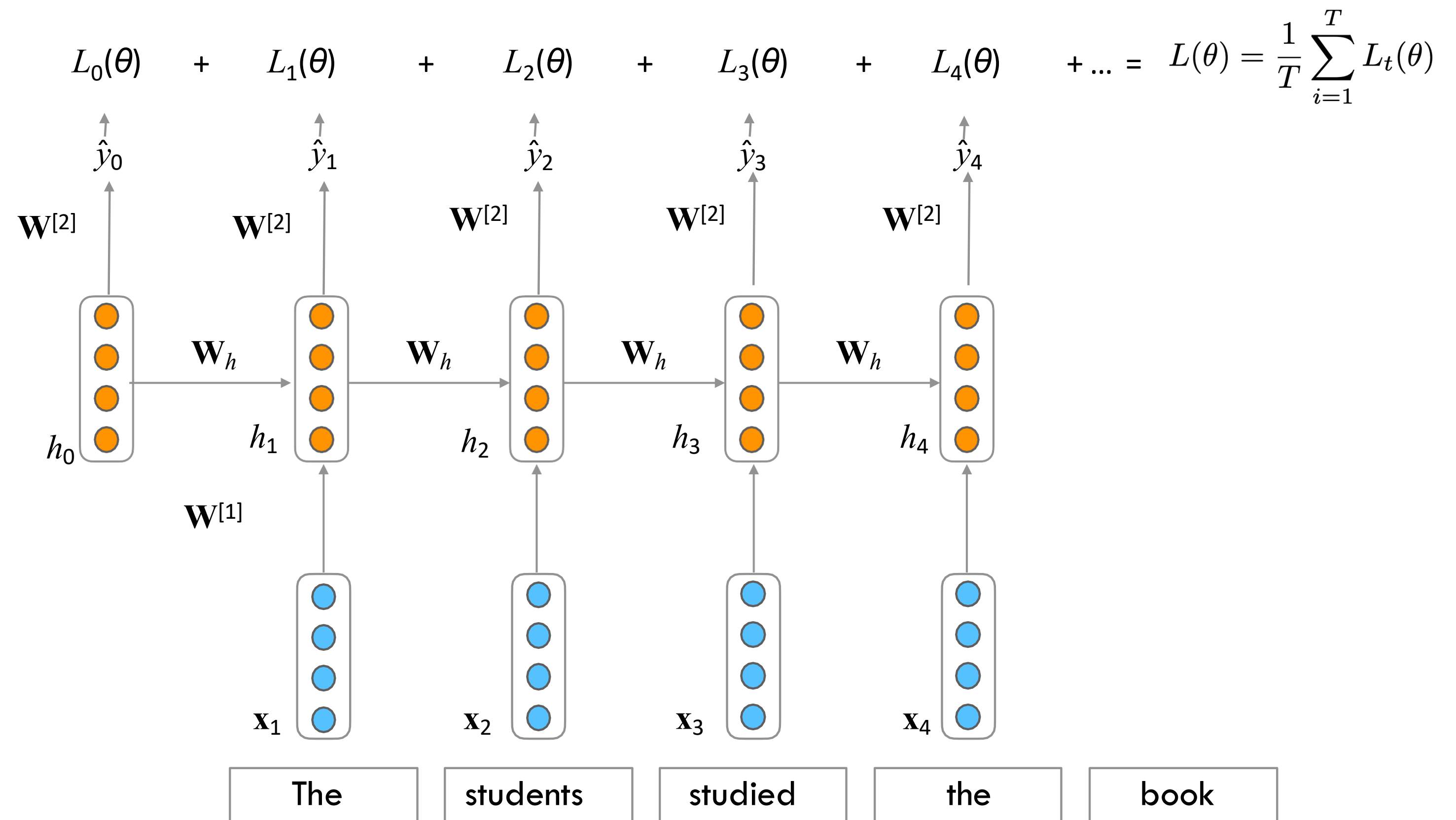
Loss



Loss

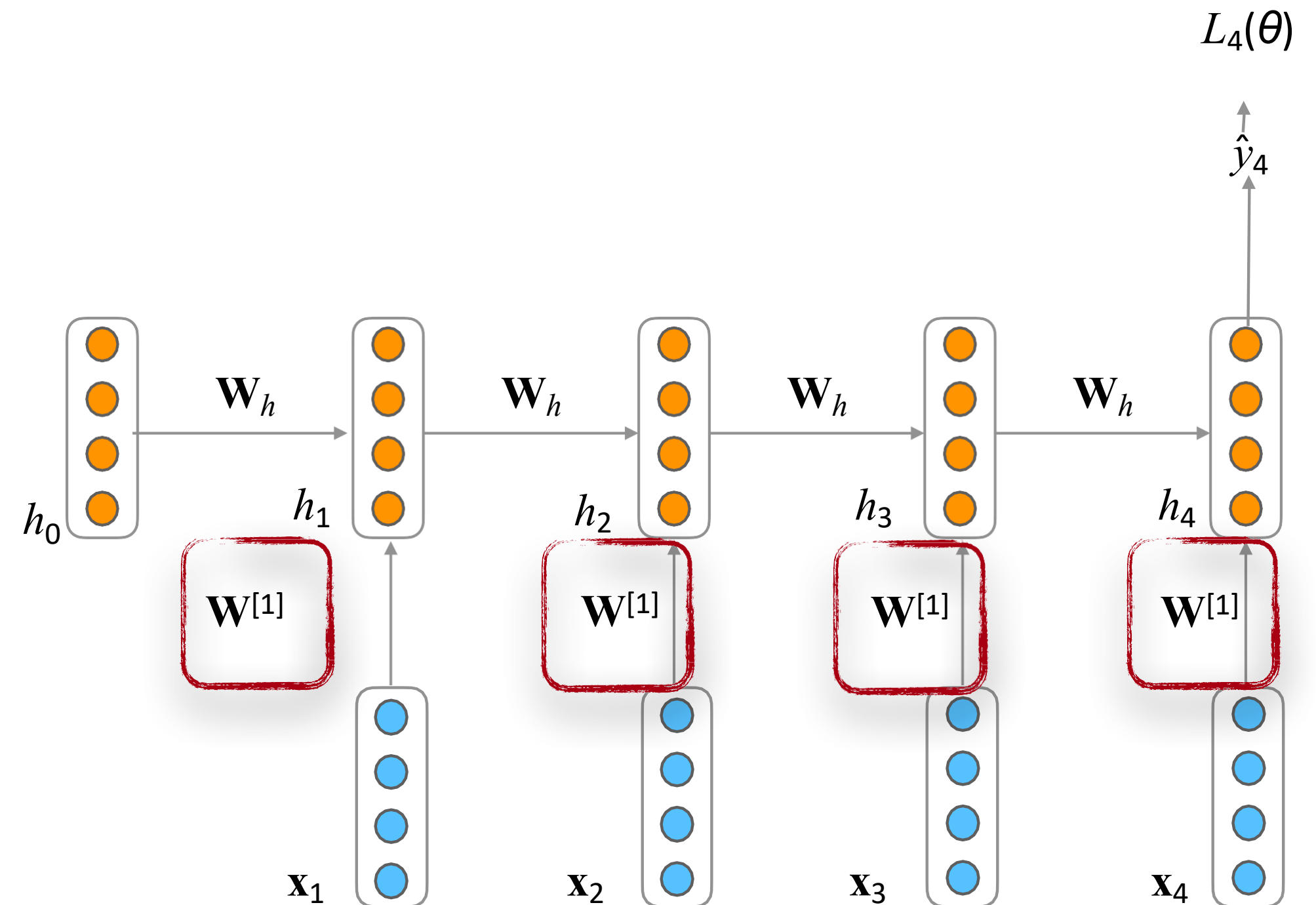


Loss

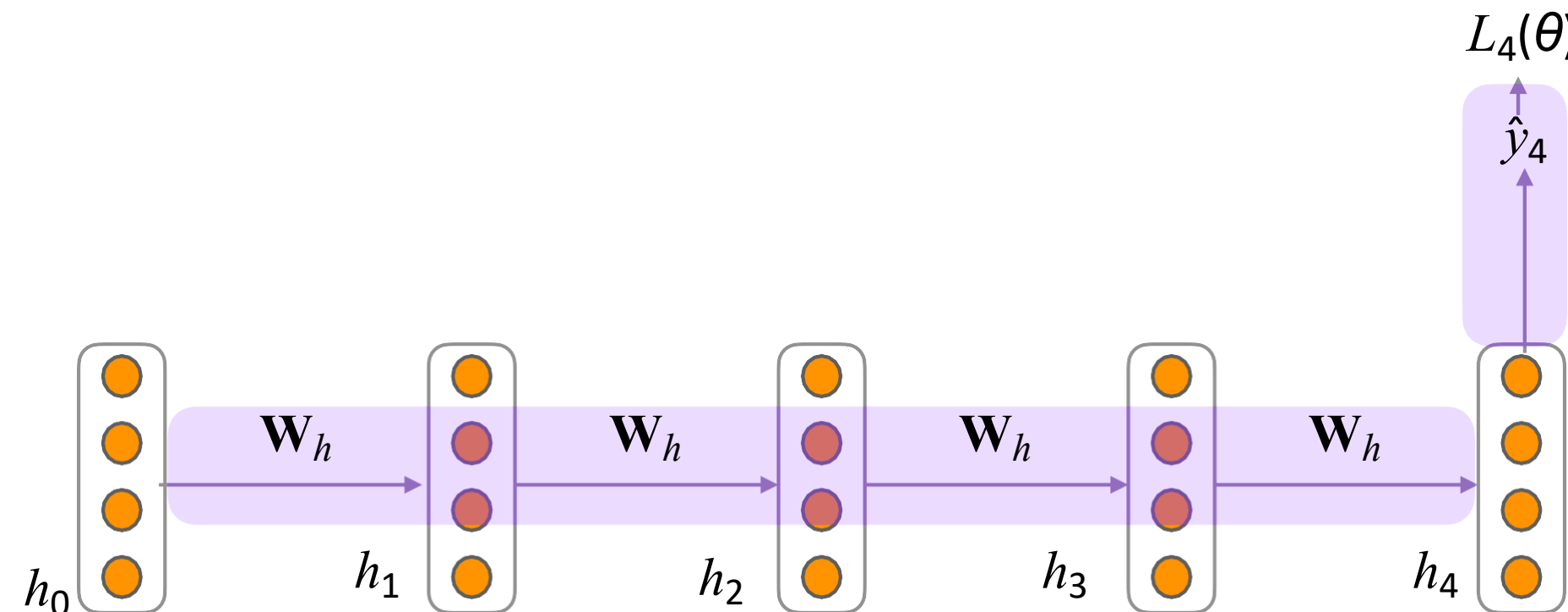


Training RNNs is hard: Gradients

- Multiply the same matrix at each time step during forward propagation
- Ideally inputs from many time steps ago can modify output
- This leads to something called the **vanishing gradient problem**



The Vanishing Gradient Problem: Intuition



When these gradients are small, the gradient signal gets smaller and smaller as it backpropagates further...

$$\begin{aligned}
 \frac{\partial L_4}{\partial h_0} &= \frac{\partial h_1}{\partial h_0} \times \frac{\partial L_4}{\partial h_1} \\
 &= \frac{\partial h_1}{\partial h_0} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial L_4}{\partial h_2} \\
 &= \frac{\partial h_1}{\partial h_0} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial L_4}{\partial h_3} \\
 &= \frac{\partial h_1}{\partial h_0} \times \frac{\partial h_2}{\partial h_1} \times \frac{\partial h_3}{\partial h_2} \times \frac{\partial h_4}{\partial h_3} \times \frac{\partial L_4}{\partial h_4}
 \end{aligned}$$

Gradient signal from far away is lost because it's much smaller than gradient signal from close-by

The Vanishing Gradient Problem: Fixes

- The main problem is that it is too difficult for the RNN to learn to preserve information over many time steps
- In a vanilla RNN, the hidden state is constantly being rewritten

$$\mathbf{h}_t = \text{ReLU}(\mathbf{W}_h \mathbf{h}_{t-1} + \mathbf{W}^{[1]} \mathbf{x}_t)$$

New design: equip an RNN with separate memory which is added to

LSTMs

Given a sequence of inputs x_t , we will compute a sequence of hidden states h_t and cell states c_t

At timestep t :

Forget gate: controls what is kept vs forgotten, from previous cell state

Input gate: controls what parts of the new cell content are written to cell

Output gate: controls what parts of cell are output to hidden state

New cell content: this is the new content to be written to the cell

Cell state: erase (“forget”) some content from last cell state, and write (“input”) some new cell content

Hidden state: read (“output”) some content from the cell

Sigmoid function: all gate values are between 0 and 1

$$f^{(t)} = \sigma \left(W_f h^{(t-1)} + U_f x^{(t)} + b_f \right)$$

$$i^{(t)} = \sigma \left(W_i h^{(t-1)} + U_i x^{(t)} + b_i \right)$$

$$o^{(t)} = \sigma \left(W_o h^{(t-1)} + U_o x^{(t)} + b_o \right)$$

$$\tilde{c}^{(t)} = \tanh \left(W_c h^{(t-1)} + U_c x^{(t)} + b_c \right)$$

$$c^{(t)} = f^{(t)} \circ c^{(t-1)} + i^{(t)} \circ \tilde{c}^{(t)}$$

$$h^{(t)} = o^{(t)} \circ \tanh c^{(t)}$$

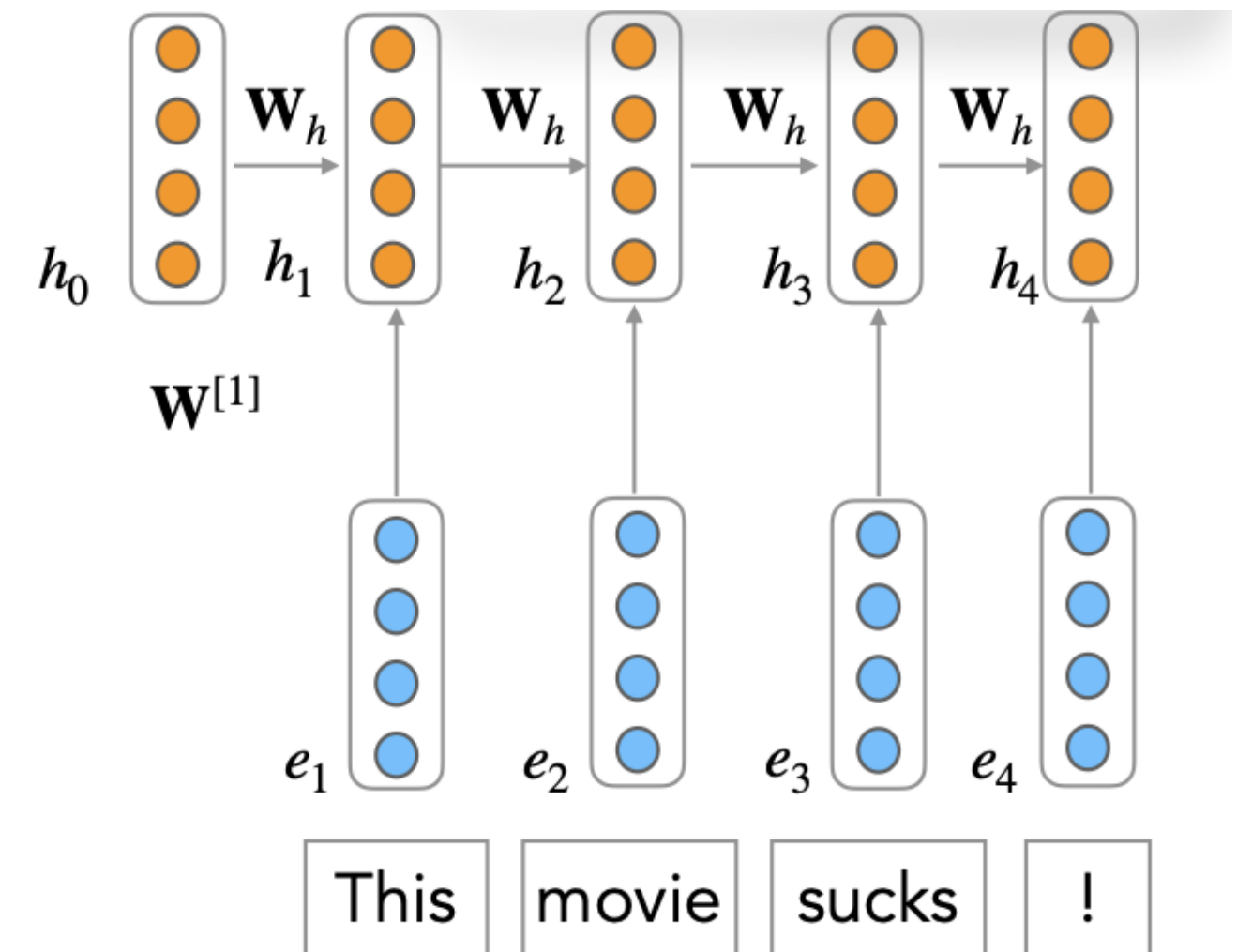
All these are vectors of same length n

Gates are applied using element-wise (or Hadamard) product: $\odot$

Applications of RNNs

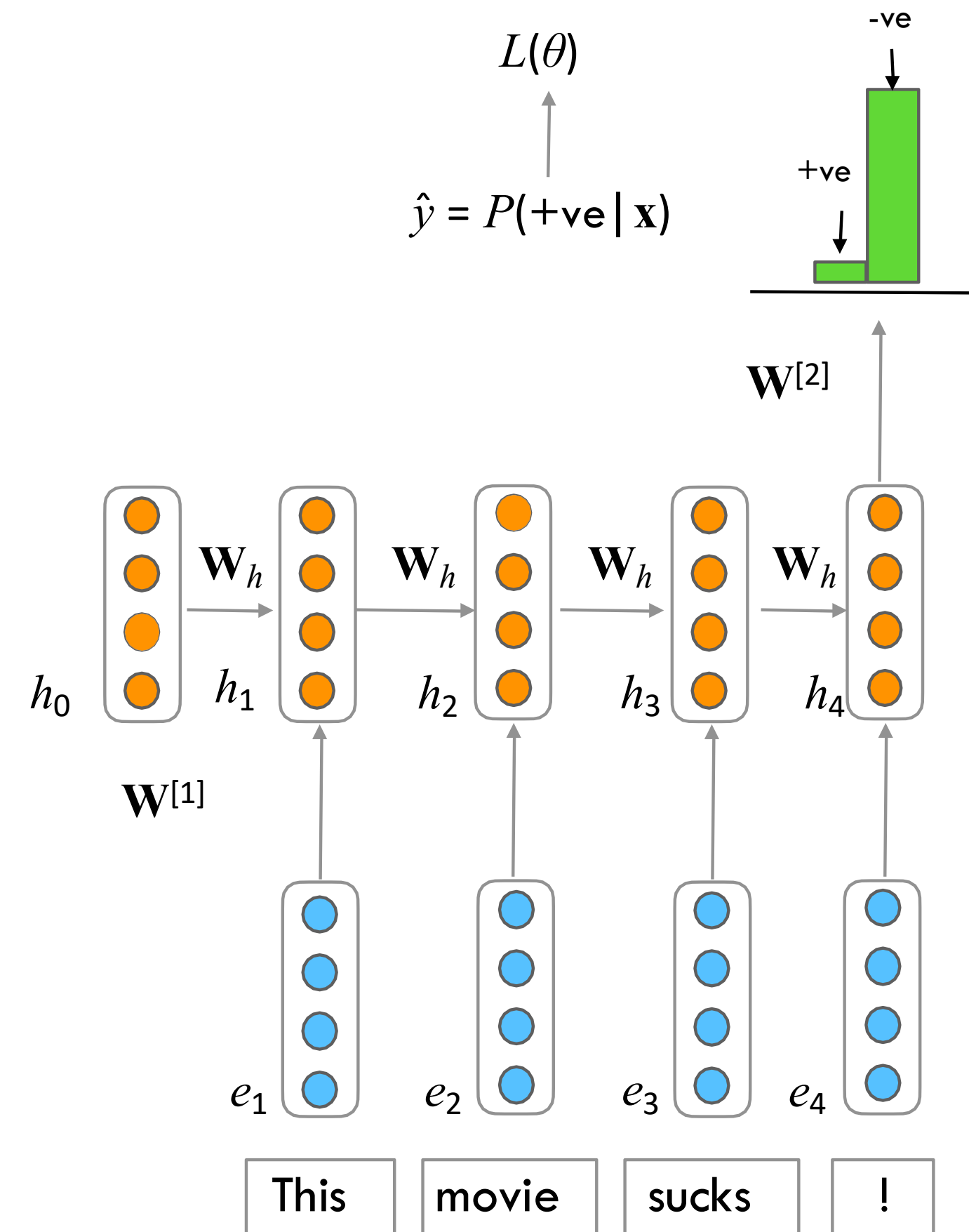
RNNs for Sequence Classification

- $\mathbf{x}$ = Entire sequence / document of length n
- $\mathbf{y}$ = (Multivariate) labels



Training RNNs for Sequence Classification

- Don't need intermediate outputs for the words in the sequence preceding the last element
- Loss function used to train the weights in the network is based entirely on the final text classification task
 - Cross-entropy loss
- Backprop: error signal from the classification is backpropagated all the way through the weights in the feedforward classifier through, to its input, and then through to the three sets of weights in the RNN



Generation with RNNLMs

- Similar to sampling from n-gram LMs
- First randomly sample a word to begin a sequence based on its suitability as the start of a sequence
- Then continue to sample words conditioned on our previous choices until
 - we reach a pre-determined length,
 - or an end of sequence token is generated

Remember sampling from n-gram LMs?

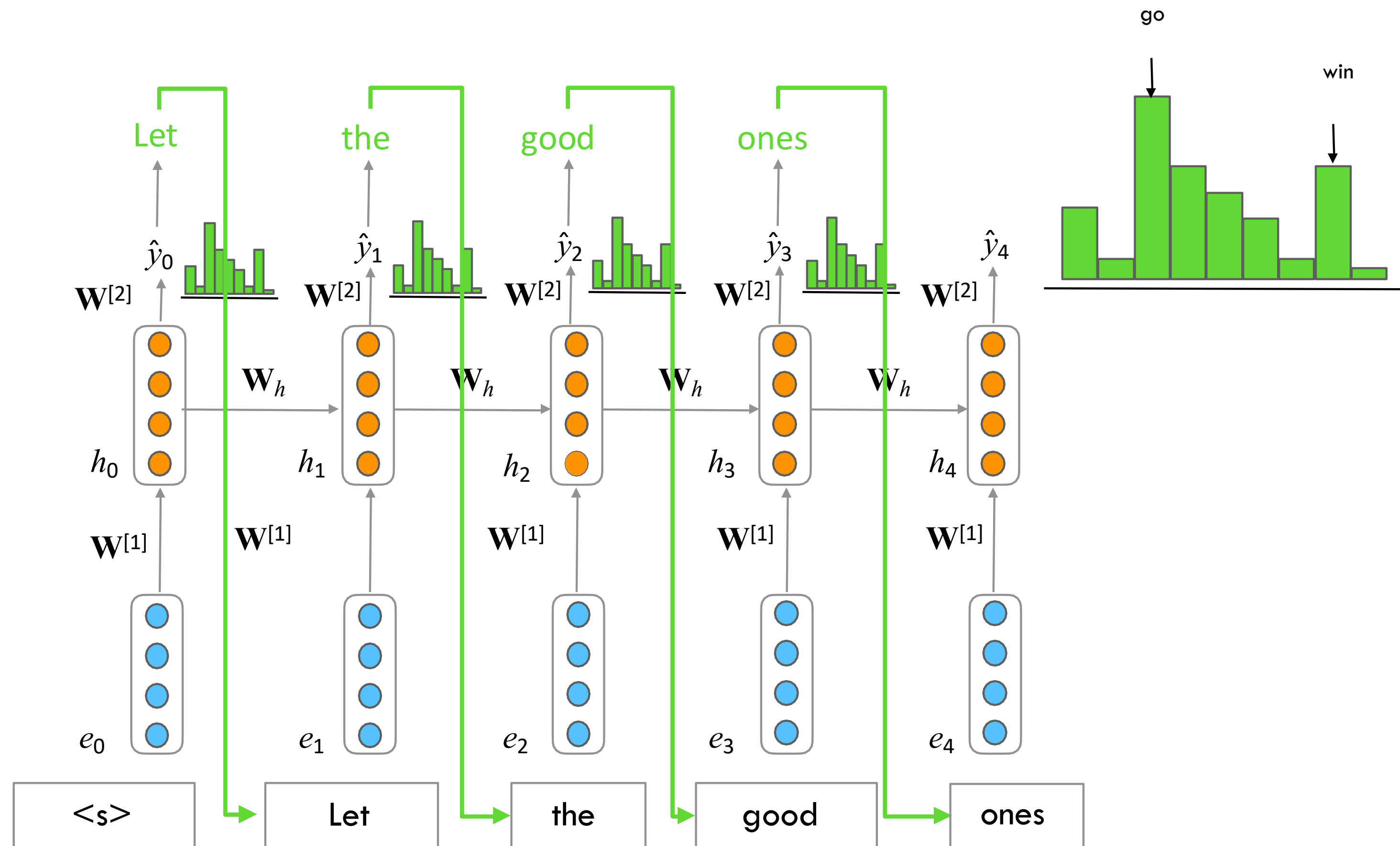
1. Choose a random bigram ($\langle s \rangle$, w_0) according to its probability
2. Now choose a random bigram (w_t , w_{t+1}) according to its probability...and so on until we choose $\langle /s \rangle$

```
<s> I
    I want
      want to
        to eat
          eat Chinese
            Chinese food
              food </s>

I want to eat Chinese food
```

$\arg \max \hat{y}_i$

$$\hat{y}_4 = P(x_5 | \text{Let the good ones})$$



Generation with RNNLMs

- Sample a word in the output from the softmax distribution that results from using the beginning of sentence marker, $\langle s \rangle$, as the first input.
- Use the word embedding for that first word as the input to the network at the next time step, and then sample the next word in the same fashion.
- Continue generating until the end of sentence marker, $\langle /s \rangle$, is sampled or a fixed length limit is reached.

Repeated sampling of the next word conditioned on previous choices

Autoregressive Generation

Lecture Outline

- Announcements
- Recap: Recurrent Neural Nets
- Seq2Seq Modeling with Encoder-Decoder Networks
- Attention Mechanism
- More on Attention

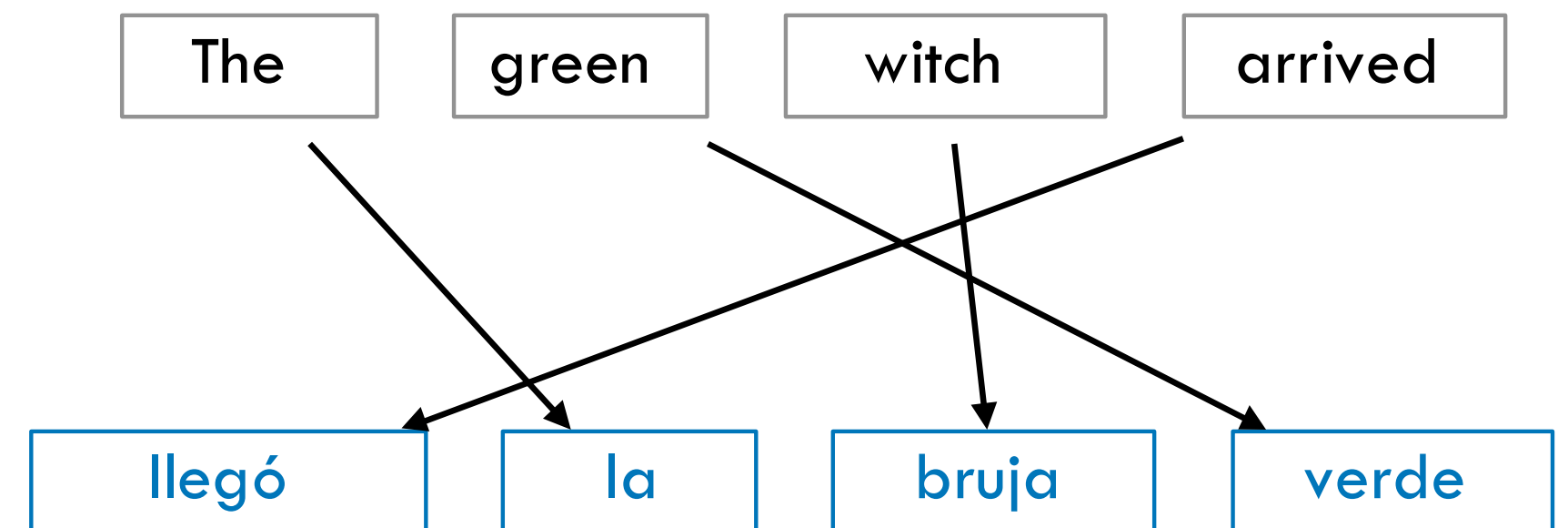
RNNLMs are Autoregressive Models

- Model that predicts a value at time t based on a function of the previous values at times $t - 1$, $t - 2$, and so on
- Word generated at each time step is conditioned on the word selected by the network from the previous step
 - State-of-the-art generation approaches are all autoregressive!
 - Machine translation, question answering, summarization
- Key technique: prime the generation with the most suitable context

Provide rich task-appropriate context!

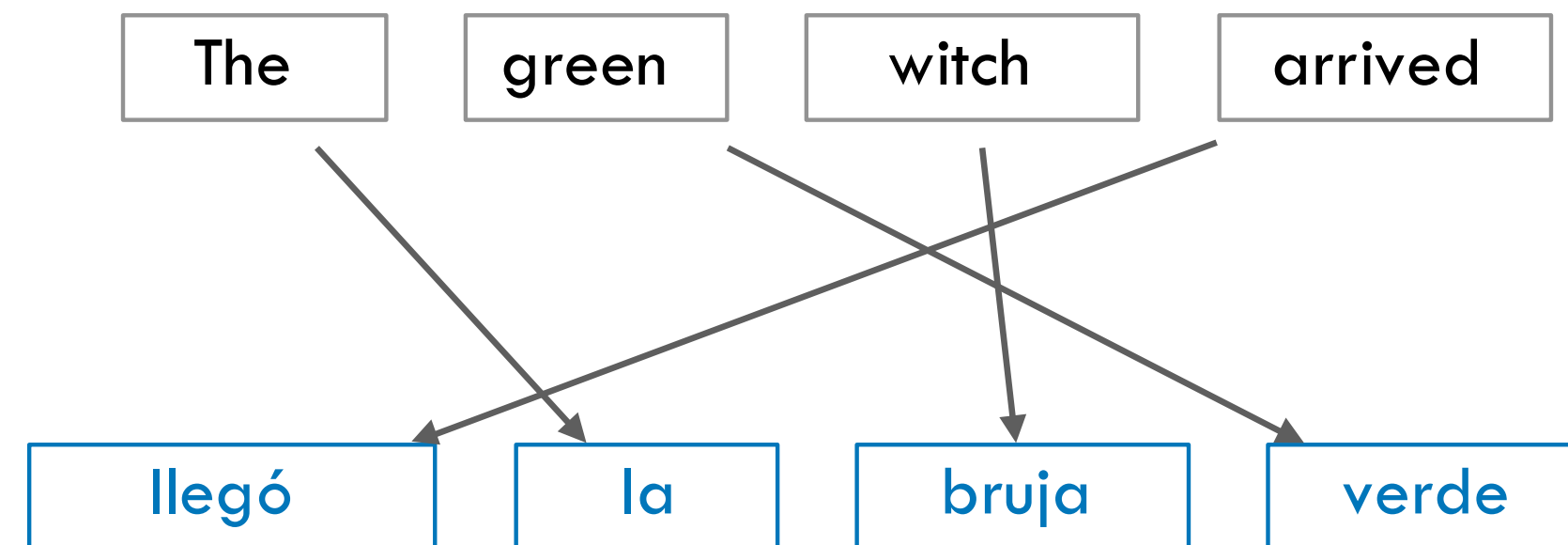
(Neural) Machine Translation

- Sequence Generation Problem (as opposed to sequence classification)
 - x = Source sequence of length n
 - y = Target sequence of length m
- Different from regular generation from an LM
 - Since we expect the target sequence to serve a specific utility (translate the source)



Sequence-to-Sequence (Seq2seq)

Sequence-to-Sequence Generation



- Mapping between a token in the input and a token in the output can be very indirect
 - in some languages the verb appears at the beginning of the sentence; e.g. Arabic, Hawaiian
 - in other languages at the end; e.g. Hindi
 - in other languages between the subject and the object; e.g. English
- Does not necessarily align in a word-word way!

Need a special architecture to summarize the entire context!

Sequence-to-Sequence Models

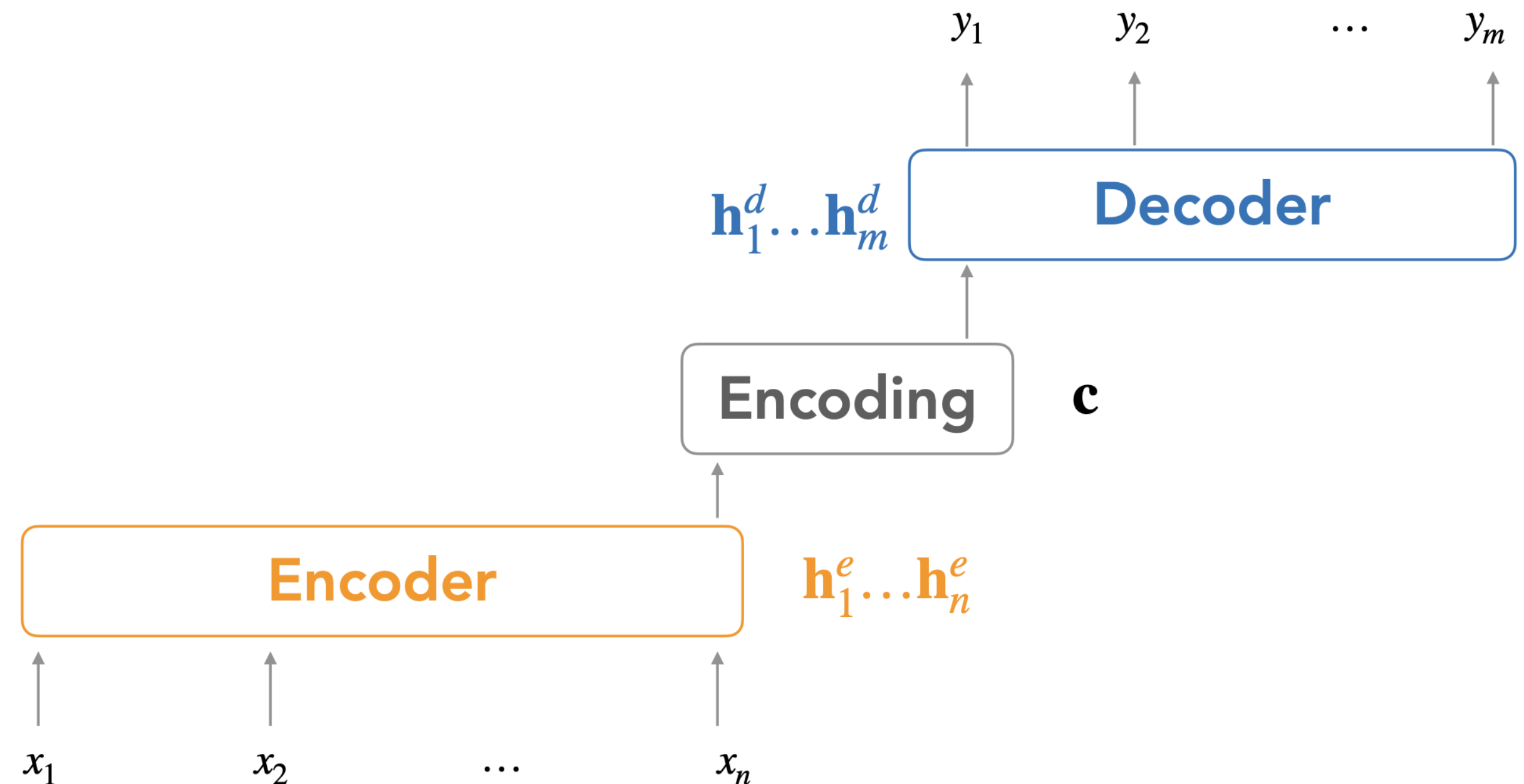
- Models capable of generating contextually appropriate, arbitrary length, output sequences given an input sequence.
- The key idea underlying these networks is the use of an **encoder network** that takes an input sequence and creates a contextualized representation of it, often called the context.
- This representation is then passed to a **decoder network** which generates a task- specific output sequence.

Encoder-Decoder Networks

Sequence-to-Sequence Modeling with Encoder-Decoder Networks

Encoder-Decoder Networks

Encoder-decoder networks consist of three components:



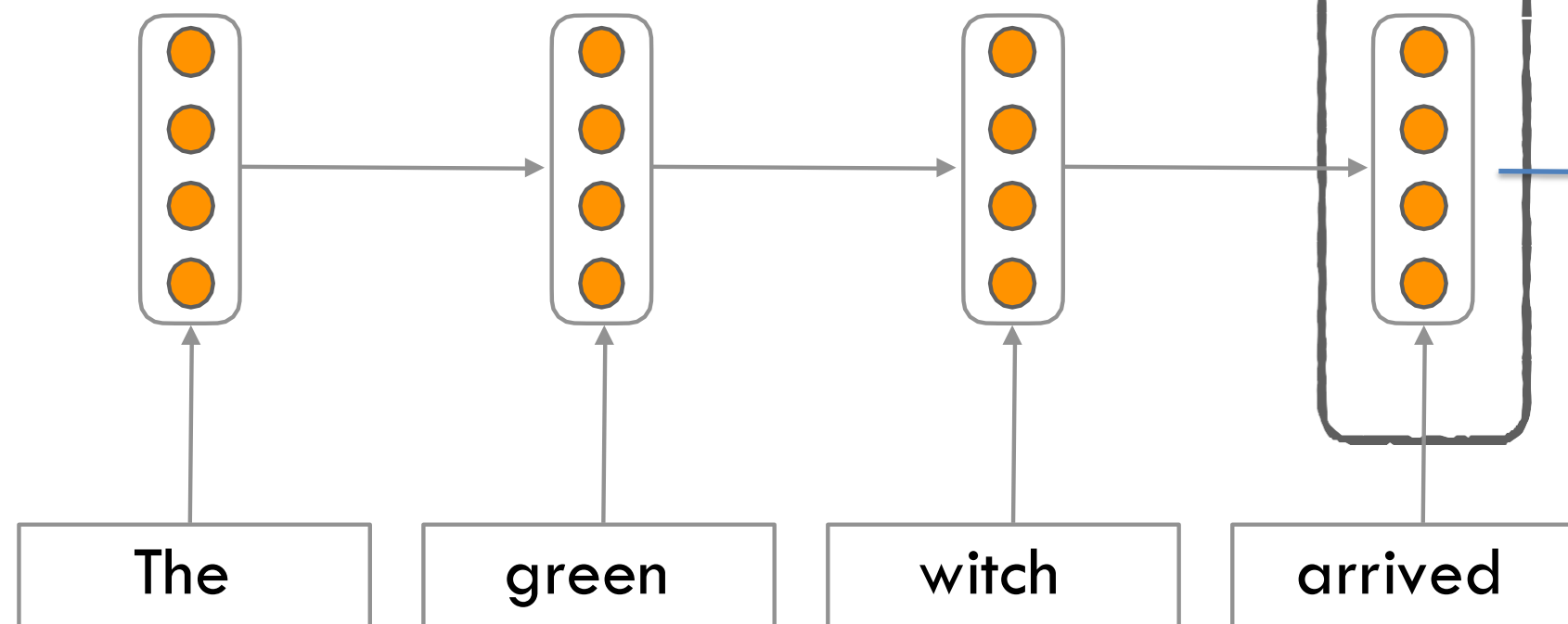
Produces an
encoding of the
source sequence

Represents input sequence.
Provides initial hidden state for
Decoder RNN

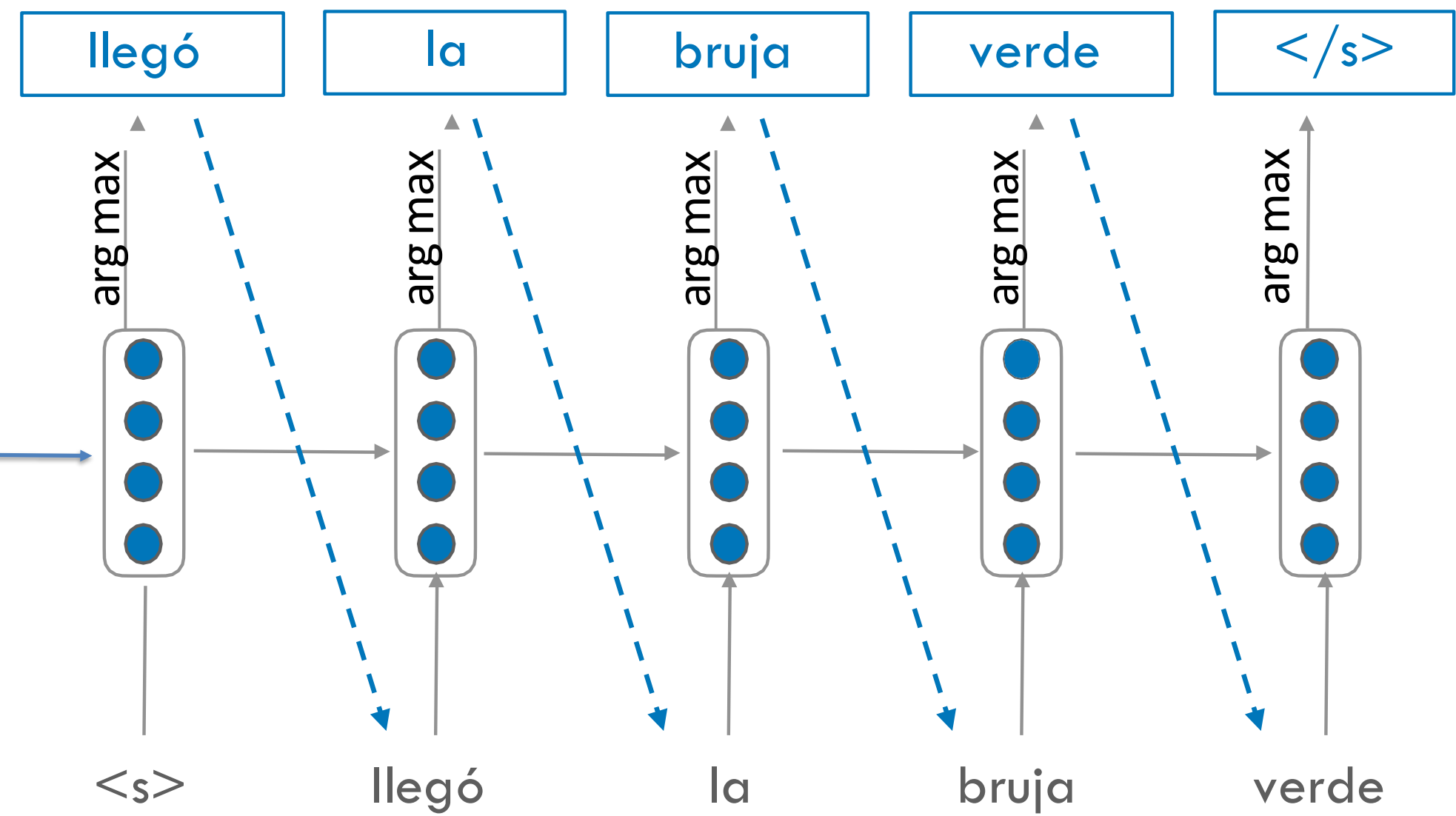
Target Sentence y

Encoding
 h_0

Encoder RNN



Source Sentence

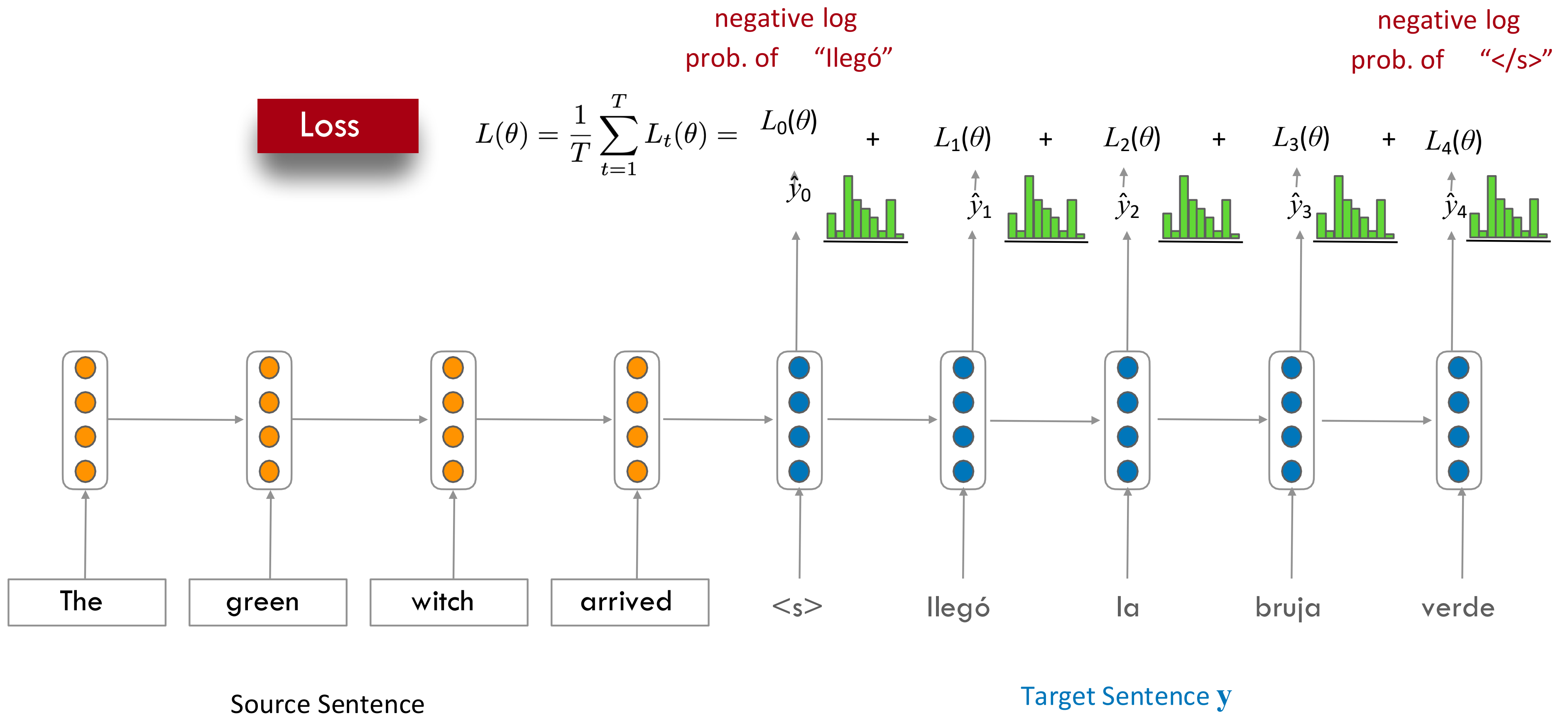


Language Model that produces the target
sentence conditioned on the encoding

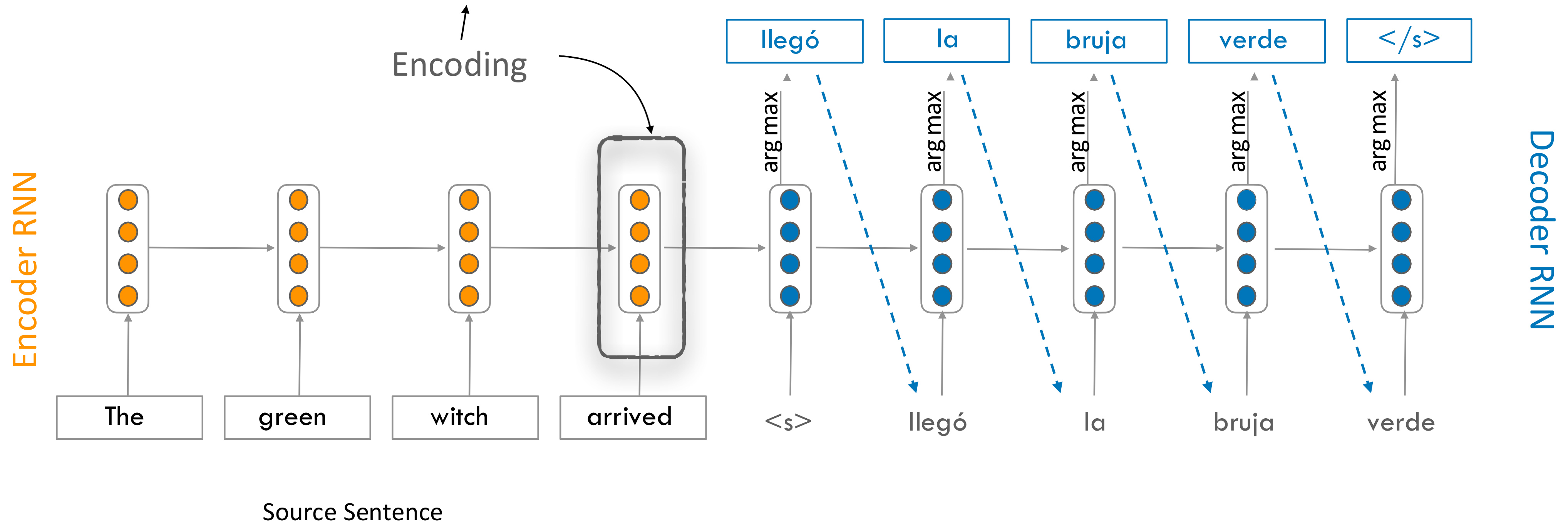
Decoder RNN

Encoder RNN

Decoder RNN



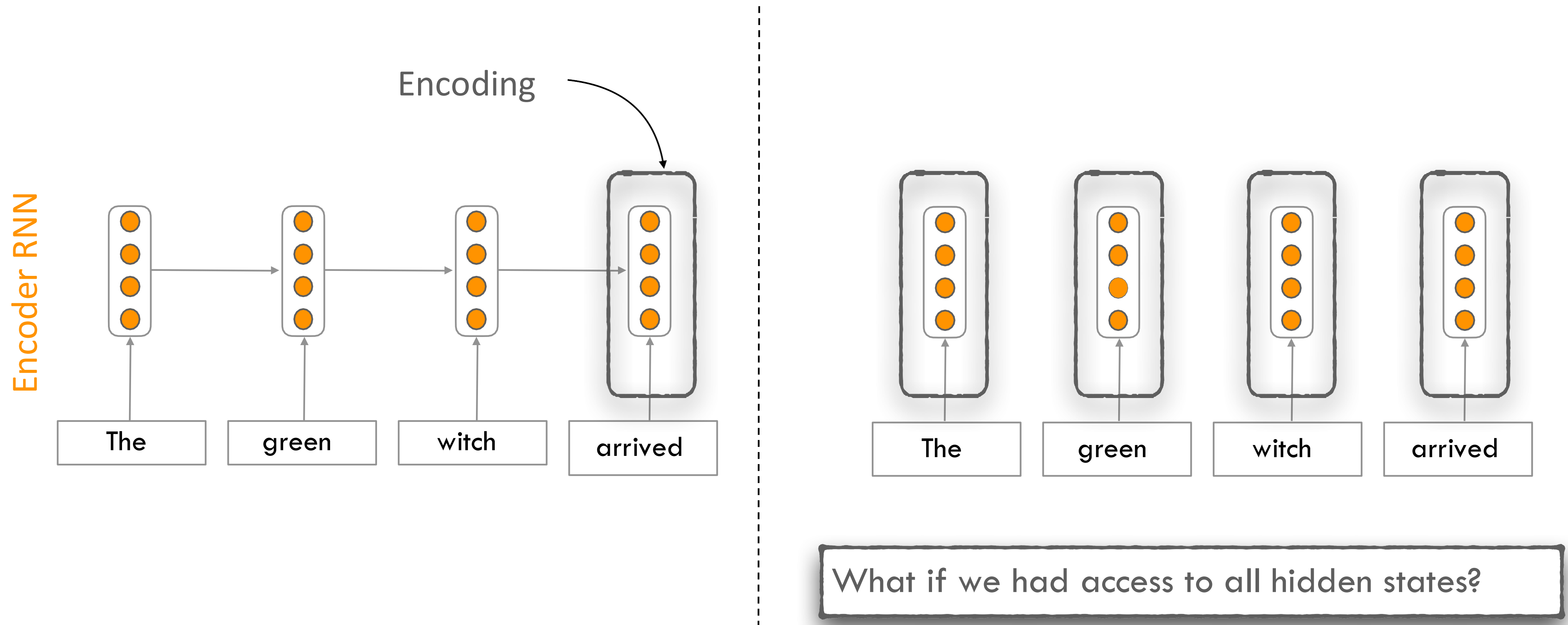
This needs to capture all information about the source sentence. **Information bottleneck!**



“you can’t cram the meaning of a whole sentence into a single vector!”

– Ray Mooney, Professor of Computer Science, UT Austin

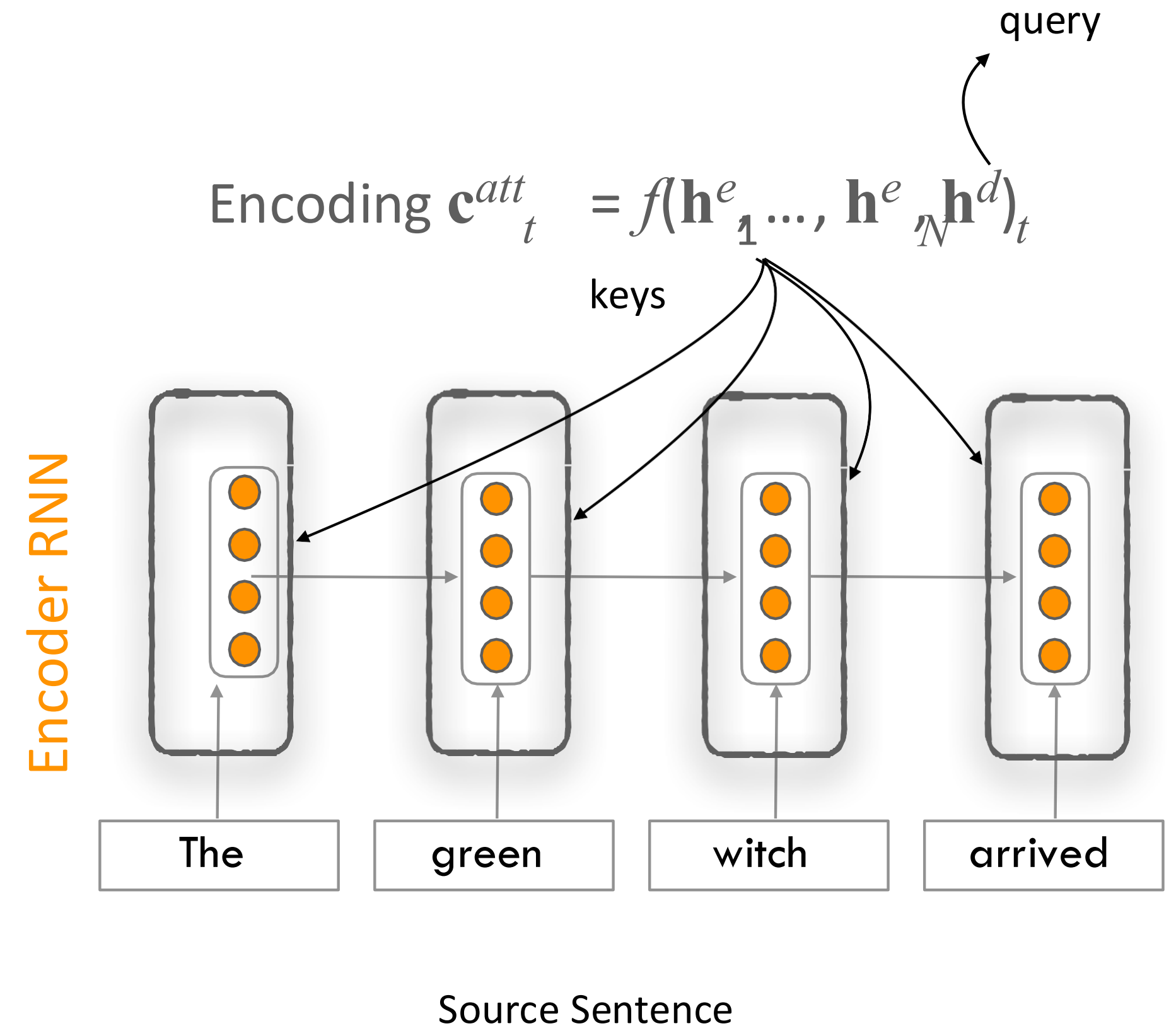
Information Bottleneck: One Solution



Attention Mechanism

Attention Mechanism

- Attention mechanisms allow the decoder to focus on a particular part of the source sequence **at each time step of decoding**
- Fixed-length vector $\mathbf{c}^{att}$ (attention context vector)
 - Take a weighted sum of all the encoder hidden states
 - One vector per time step of the decoder!
 - Weights attend to part of the source text relevant for the token the decoder is producing at step
- In general, we have a single query vector and multiple key vectors.
 - We want to score each query-key pair

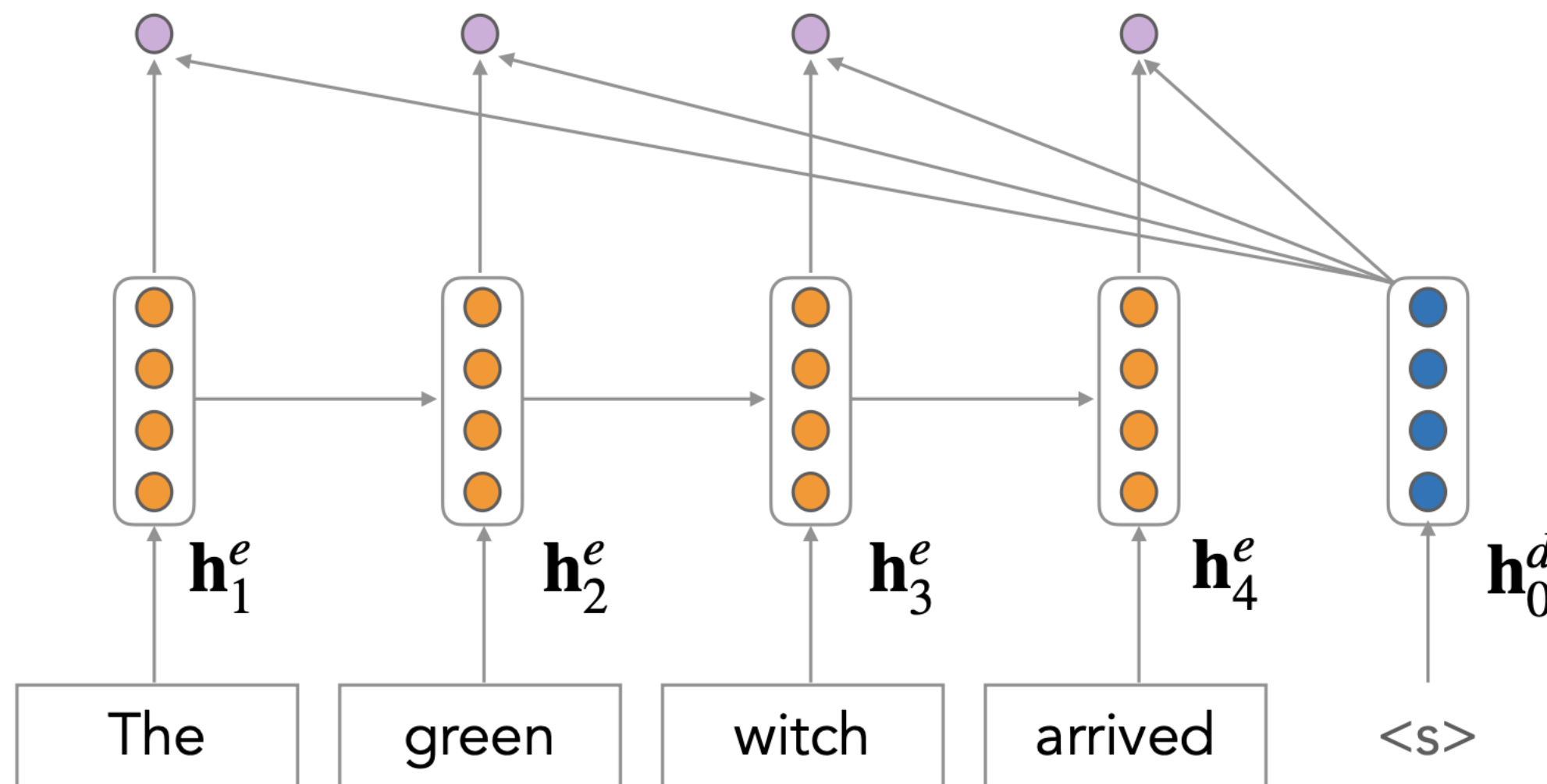


Seq2Seq with Attention

$$\text{score}(\mathbf{h}_t^d, \mathbf{h}_j^e) = \mathbf{h}_t^d \cdot \mathbf{h}_j^e$$

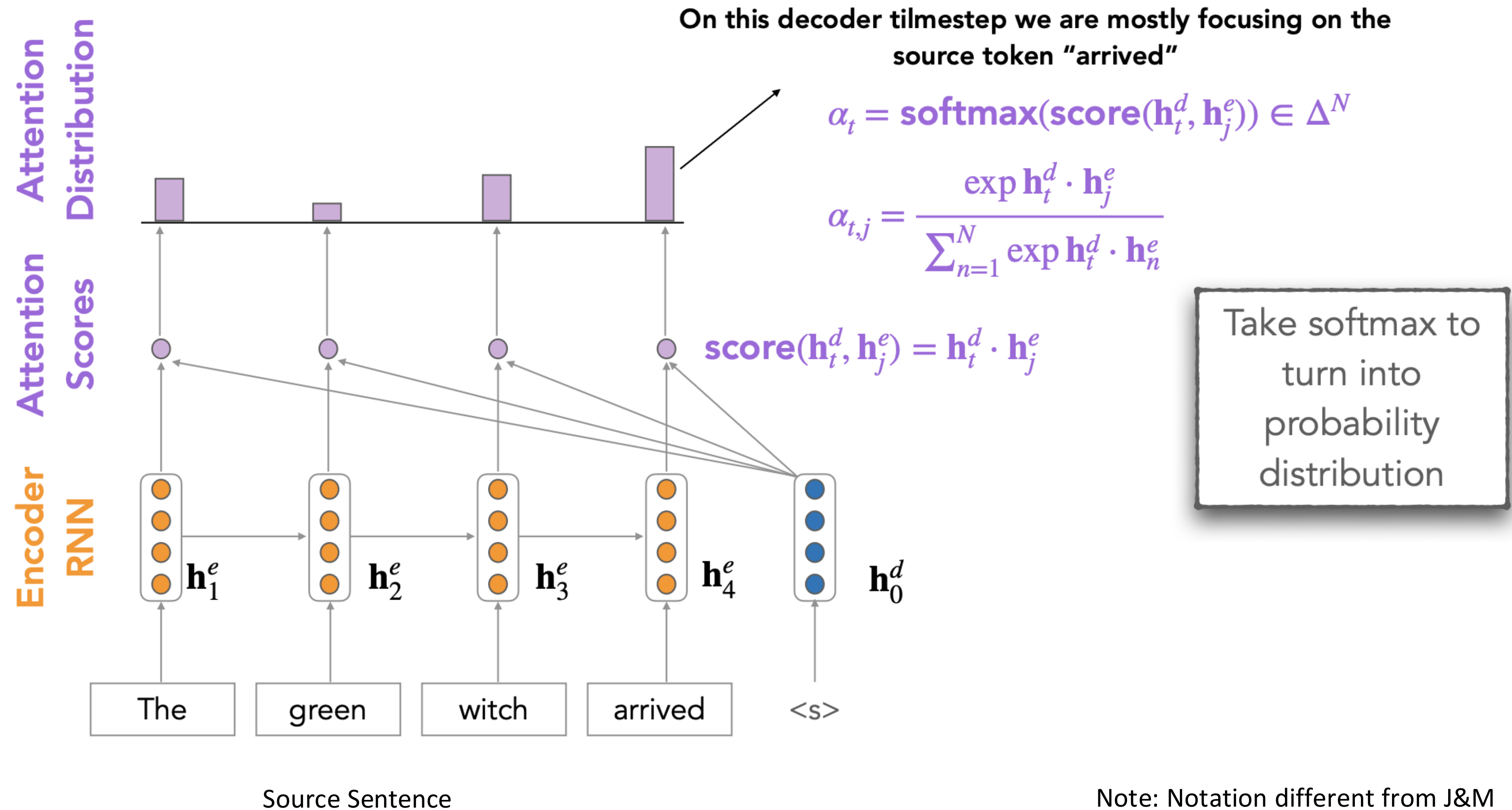
Dot product with keys (encoder hidden states) to encode similarity with what is decoded so far...

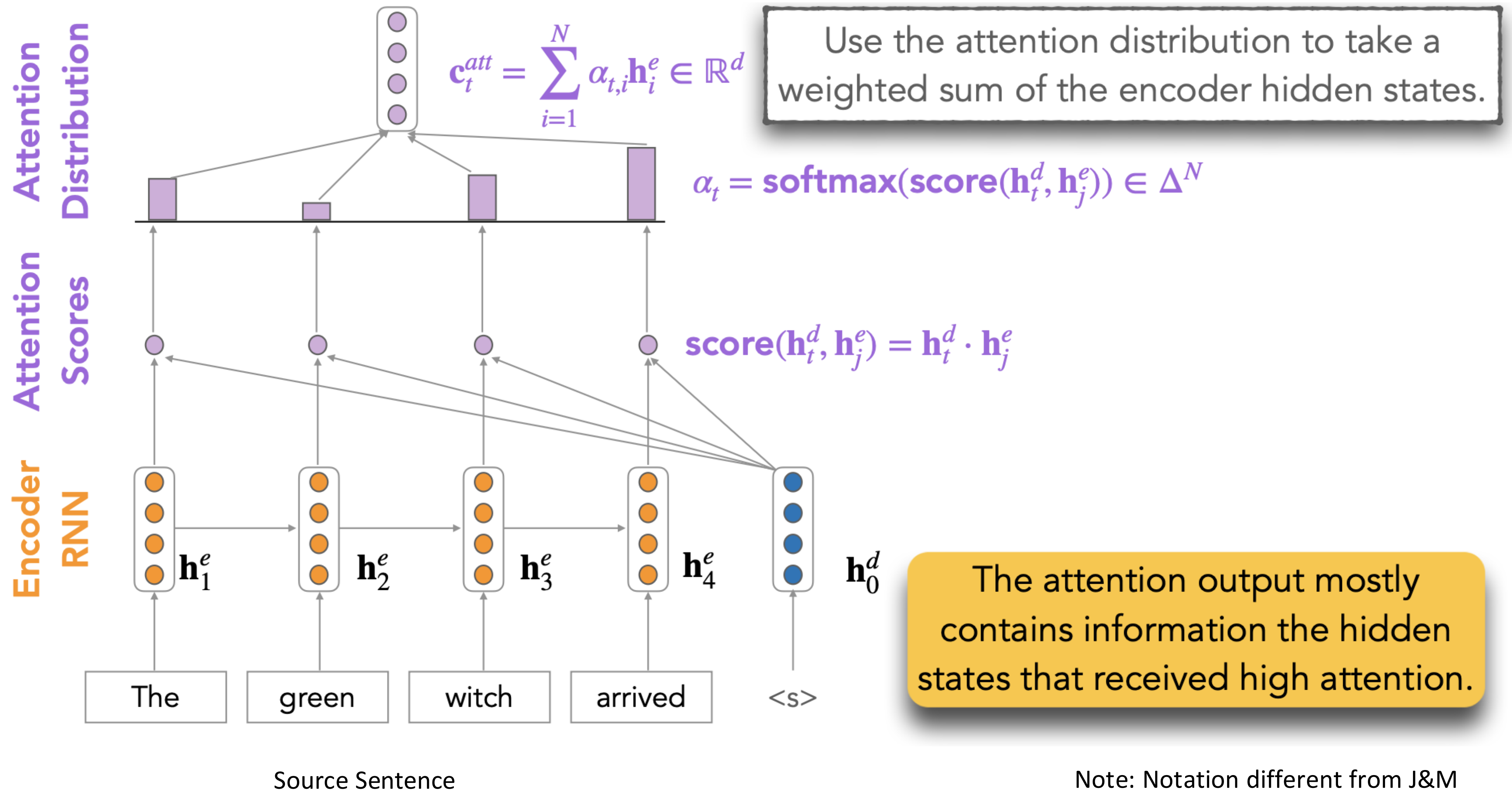
Query 1: Decoder, first time step

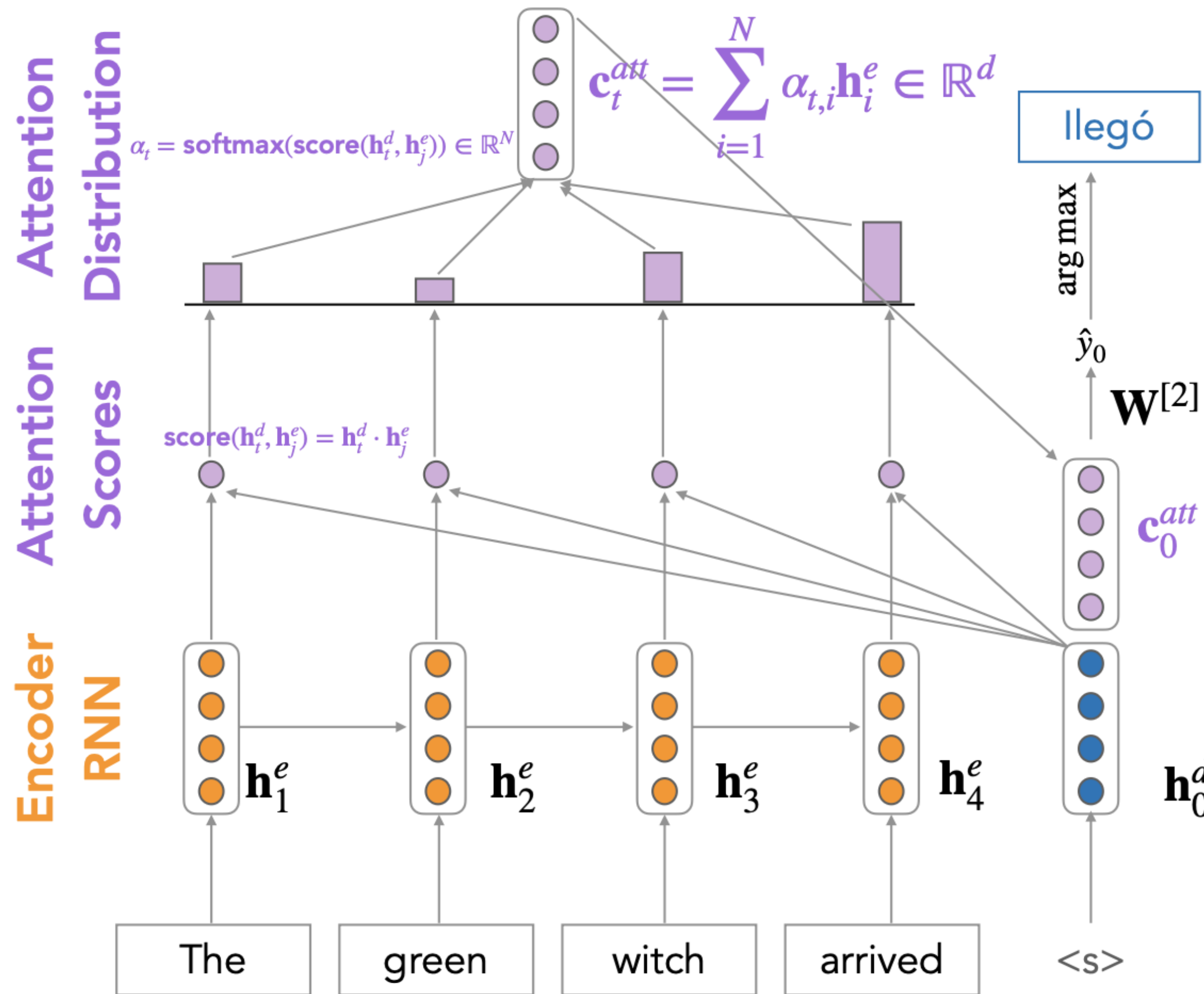


Source Sentence

Dot product attention



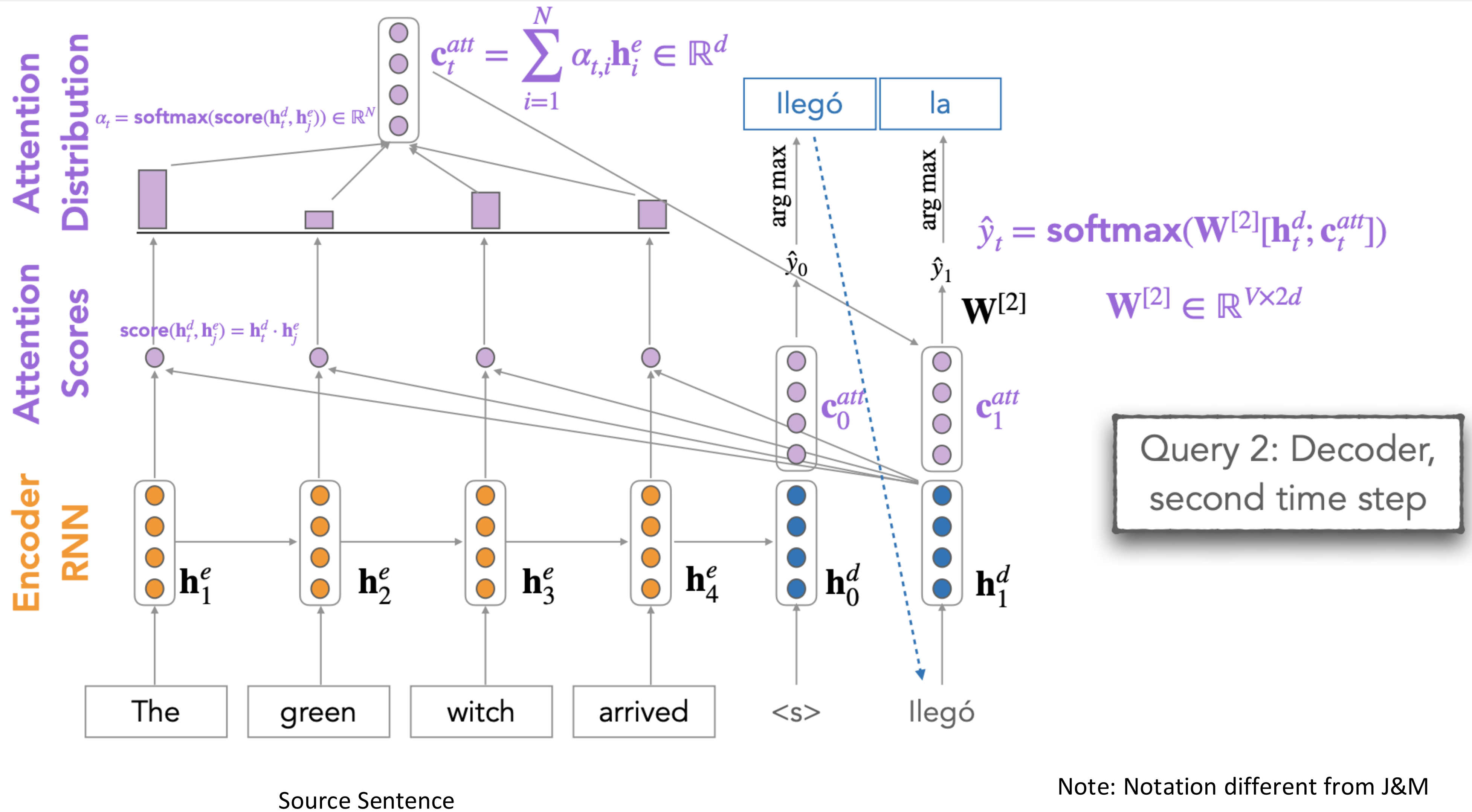




$$\hat{y}_t = \text{softmax}(\mathbf{W}^{[2]}[\mathbf{h}_t^d; \mathbf{c}_t^{\text{att}}])$$

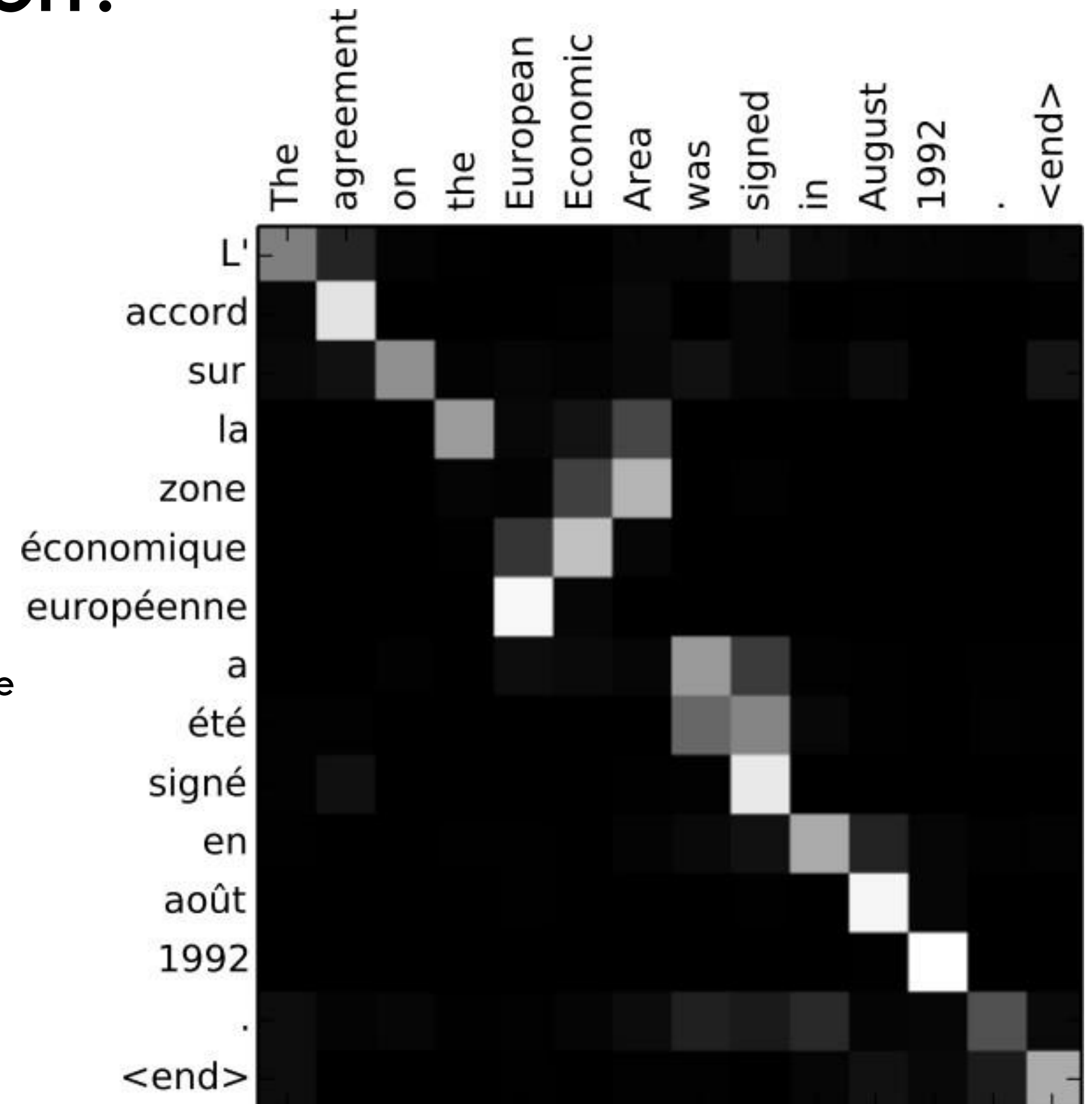
$$\mathbf{W}^{[2]} \in \mathbb{R}^{V \times 2d}$$

Concatenate attention output with decoder hidden state, then use to compute $\hat{y}_0$ as before

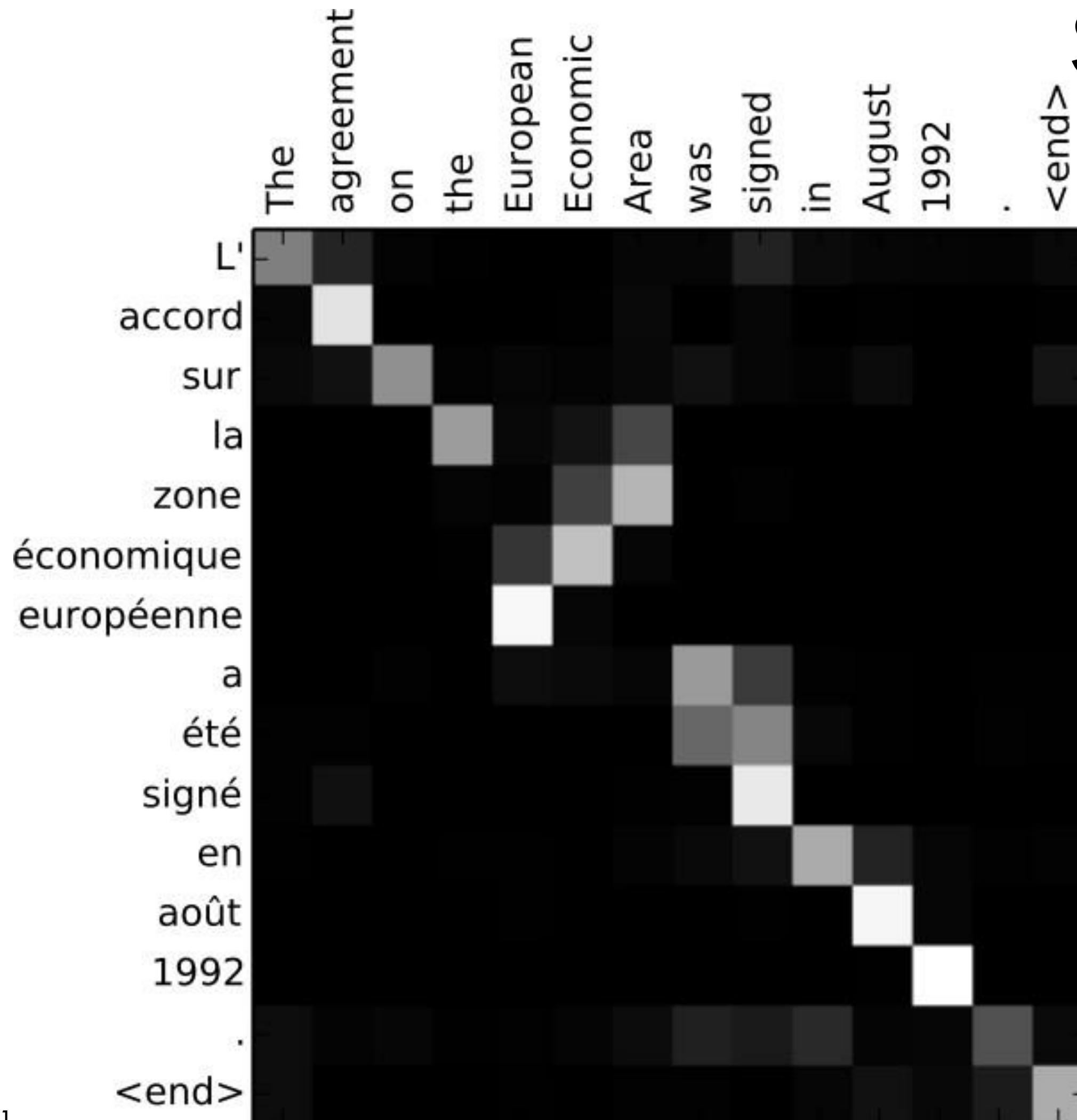


Why Attention?

- Attention significantly improves neural machine translation performance
 - Very useful to allow decoder to focus on certain parts of the source
- Attention solves the information bottleneck problem
 - Attention allows decoder to look directly at source; bypass bottleneck
- Attention helps with vanishing gradient problem
 - Provides shortcut to faraway states
- Attention provides some interpretability
 - By inspecting attention distribution, we can see what the decoder was focusing on →
 - We get alignment for free! We never explicitly trained an alignment system! The network just learned alignment by itself



Seq2Seq Summary



- Seq2Seq modeling is popular for close-ended generation tasks
 - MT, Summarization, QA
 - Involves an encoder and a decoder
 - Can be any neural architecture!
- Popular Seq2Seq Models using Transformers: BART, T5
- Secret Sauce: Attention
- Next: Self-Attention and Transformers